IF3070 DASAR INTELIGENSI ARTIFISIAL

TUGAS BESAR 1

Pencarian Solusi Penempatan Kendaraan pada Geladak Kapal dengan Local Search



Disusun oleh:

Ibrahim Ferizarizqi Permana	18224120
Adham Sachadeva Purwadi	18224030
Muhammad Reyna Athallah Agoes	18224112

PRODI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

DAFTAR ISI

BAB I DESKRIPSI PERSOALAN.....5

BAB II PEMBAHASAN.....7

2.1 Penjelasan Objective Function..... 7

2.1.1 Objective Function..... 7

2.1.2 Source Code Objective Function.....8

2.1.3 Kesimpulan Pemilihan Objective Function.....8

2.2 Penjelasan Pemilihan Initial State..... 8

a. Source Code Random Initial State.....8

b. Penjelasan Random Initial State.....9

2.3 Penjelasan Penerapan Constraints..... 10

a. Source Code Penerapan Constraints..... 10

b. Penjelasan Penerapan Constraints..... 10

c. Pembentukan Neighbor.....11

2.4 Penjelasan Implementasi Algoritma Local Search.....12

2.4.1 Stochastic Hill-Climbing..... 12

2.4.2 Simulated Annealing..... 13

2.4.3 Genetic Algorithm..... 15

2.5 Hasil Eksperimen dan Analisis.....19

2.5.1 Stochastic Hill-Climbing..... 20

2.5.2 Simulated Annealing..... 23

2.5.3 Genetic Algorithm..... 27

2.6 Analisis Perbandingan Algoritma dalam Mendekati Global Optima.....45

2.7 Analisis Perbandingan Hasil Pencarian Algoritma..... 46

2.8 Analisis Perbandingan Durasi Pencarian Algoritma..... 47

2.9 Analisis Perbandingan Konsistensi Algoritma..... 47
BAB III KESIMPULAN DAN SARAN..... 49
3.1 Kesimpulan..... 49
3.2 Saran..... 49
BAB IV PEMBAGIAN TUGAS TIAP ANGGOTA KELOMPOK..... 50
REFERENSI..... 51
LAMPIRAN A DATA UJI DAN PENGGUNAAN AI..... 52
LAMPIRAN B RINCIAN STATE PERCOBAAN..... 53
LAMPIRAN C REPRODUKSI EKSPERIMEN..... 77

DAFTAR GAMBAR

Gambar 2.1 Stochastic Hill-Climbing HC Percobaan Pertama..... 20
Gambar 2.2 Stochastic Hill-Climbing HC Percobaan Kedua.....21
Gambar 2.3 Stochastic Hill-Climbing HC Percobaan Ketiga.....22
Gambar 2.4 Simulated Annealing SA Percobaan Pertama.....23
Gambar 2.5 Probabilitas Penerimaan dan Temperatur SA Seed 42.....23
Gambar 2.6 Simulated Annealing SA Percobaan Kedua.....24
Gambar 2.7 Probabilitas Penerimaan dan Temperatur SA Seed 43.....24
Gambar 2.8 Simulated Annealing SA Percobaan Ketiga..... 25
Gambar 2.9 Probabilitas Penerimaan dan Temperatur SA Seed 44.....25
Gambar 2.10 Genetic Algorithm GA_G20 Percobaan Pertama..... 27
Gambar 2.11 Genetic Algorithm GA_G20 Percobaan Kedua.....28
Gambar 2.12 Genetic Algorithm GA_G20 Percobaan Ketiga.....29
Gambar 2.13 Genetic Algorithm GA_G60 Percobaan Pertama..... 30
Gambar 2.14 Genetic Algorithm GA_G60 Percobaan Kedua.....31
Gambar 2.15 Genetic Algorithm GA_G60 Percobaan Ketiga.....32
Gambar 2.16 Genetic Algorithm GA_G120 Percobaan Pertama..... 33
Gambar 2.17 Genetic Algorithm GA_G120 Percobaan Kedua.....34
Gambar 2.18 Genetic Algorithm GA_G120 Percobaan Ketiga.....35
Gambar 2.19 Genetic Algorithm GA_P10 Percobaan Pertama.....36
Gambar 2.20 Genetic Algorithm GA_P10 Percobaan Kedua..... 37
Gambar 2.21 Genetic Algorithm GA_P10 Percobaan Ketiga..... 38
Gambar 2.22 Genetic Algorithm GA_P30 Percobaan Pertama.....39
Gambar 2.23 Genetic Algorithm GA_P30 Percobaan Kedua..... 40
Gambar 2.24 Genetic Algorithm GA_P30 Percobaan Ketiga..... 41
Gambar 2.25 Genetic Algorithm GA_P60 Percobaan Pertama.....42
Gambar 2.26 Genetic Algorithm GA_P60 Percobaan Kedua..... 43
Gambar 2.27 Genetic Algorithm GA_P60 Percobaan Ketiga..... 44

DAFTAR TABEL

Tabel 1.1 Daftar Variabel Permasalahan.....5
Tabel 2.1 Konfigurasi Eksperimen..... 19
Tabel 2.2 Rekapitulasi Hasil Pencarian..... 46
Tabel 4.1 Pembagian Tugas Kelompok..... 50
Tabel A.1 Rincian Data Uji Kendaraan..... 52
Tabel B.1 Penempatan Kendaraan HC Seed 42.....53
Tabel B.2 Penempatan Kendaraan HC Seed 43.....54
Tabel B.3 Penempatan Kendaraan HC Seed 44.....55
Tabel B.4 Penempatan Kendaraan SA Seed 42..... 56
Tabel B.5 Penempatan Kendaraan SA Seed 43..... 57
Tabel B.6 Penempatan Kendaraan SA Seed 44..... 58
Tabel B.7 Penempatan Kendaraan GA_G20 Seed 42.....59
Tabel B.8 Penempatan Kendaraan GA_G20 Seed 43.....60
Tabel B.9 Penempatan Kendaraan GA_G20 Seed 44.....61
Tabel B.10 Penempatan Kendaraan GA_G60 Seed 42.....62
Tabel B.11 Penempatan Kendaraan GA_G60 Seed 43.....63
Tabel B.12 Penempatan Kendaraan GA_G60 Seed 44.....64
Tabel B.13 Penempatan Kendaraan GA_G120 Seed 42.....65
Tabel B.14 Penempatan Kendaraan GA_G120 Seed 43.....66
Tabel B.15 Penempatan Kendaraan GA_G120 Seed 44.....67
Tabel B.16 Penempatan Kendaraan GA_P10 Seed 42..... 68
Tabel B.17 Penempatan Kendaraan GA_P10 Seed 43..... 69
Tabel B.18 Penempatan Kendaraan GA_P10 Seed 44..... 70
Tabel B.19 Penempatan Kendaraan GA_P30 Seed 42..... 71
Tabel B.20 Penempatan Kendaraan GA_P30 Seed 43..... 72
Tabel B.21 Penempatan Kendaraan GA_P30 Seed 44..... 73
Tabel B.22 Penempatan Kendaraan GA_P60 Seed 42..... 74
Tabel B.23 Penempatan Kendaraan GA_P60 Seed 43..... 75
Tabel B.24 Penempatan Kendaraan GA_P60 Seed 44..... 76

BAB I
DESKRIPSI PERSOALAN

Pencarian solusi penempatan kendaraan pada geladak kapal merupakan salah satu permasalahan optimisasi kombinatorial. Pada permasalahan ini, sekumpulan kendaraan dengan ukuran, berat, dan tarif pengiriman yang berbeda harus dipilih serta ditempatkan pada sebuah geladak kapal. Tujuan utama dari pencarian adalah memperoleh konfigurasi yang menghasilkan total tarif pengiriman terbesar dengan tetap memenuhi batasan ruang dan kapasitas berat kapal.

Studi kasus yang digunakan adalah kegiatan pengiriman kendaraan pada PT Owen Intim Line. Perusahaan menerima kendaraan dari beberapa dealer, sedangkan ruang geladak dan kapasitas angkut yang tersedia bersifat terbatas. Oleh karena itu, tidak semua kendaraan harus diangkut. Kendaraan yang dipilih harus memiliki posisi dan orientasi yang sesuai sehingga tidak saling bertumpuk maupun berada di luar batas geladak.

Pencarian solusi dilakukan menggunakan pendekatan *local search*, yaitu *Stochastic Hill-Climbing*, *Simulated Annealing*, dan *Genetic Algorithm*. Ketiga algoritma tersebut menggunakan representasi *state* dan fungsi objektif yang sama. Perbedaan cara membentuk serta menerima kandidat solusi selanjutnya digunakan sebagai dasar untuk membandingkan hasil pencarian, waktu komputasi, dan konsistensi percobaan

Tabel 1.1 Daftar Variabel Permasalahan

Entitas	**Atribut**	**Contoh**	**Keterangan**
Vehicle	id	V01	Identitas unik kendaraan
Vehicle	Dimensions	w=2, l=3	Lebar dan panjang dalam satuan grid
Vehicle	Orientation	Horizontal	Orientasi horizontal atau vertikal
Vehicle	ShippingFee	80	Tarif dalam juta rupiah
Vehicle	Weight	5	Berat kendaraan dalam satuan unit
Vehicle	ETA	1	Perkiraan kedatangan dalam sisa hari
Ship	Dimensions	w=8, l=12	Ukuran geladak kapal
Ship	MaxCapacity	55	Batas berat seluruh muatan

Terdapat beberapa ketentuan representasi yang perlu diperhatikan. Kendaraan dan geladak direpresentasikan sebagai grid dua dimensi berbentuk persegi panjang. Koordinat

kendaraan berupa bilangan bulat, sedangkan orientasi kendaraan terdiri atas horizontal dan vertikal. Kendaraan yang tidak diangkut tetap disimpan dalam *state* dengan kedua koordinat bernilai None.

Langkah perpindahan yang digunakan meliputi memindahkan satu kendaraan, menukar posisi dua kendaraan, dan mengubah orientasi kendaraan. Setiap kandidat yang diterima harus memenuhi batas geladak, bebas tumpang tindih, dan tidak menyebabkan total berat melebihi kapasitas. ETA disimpan sebagai atribut masukan, tetapi tidak digunakan dalam fungsi objektif pada implementasi ini.

BAB II
PEMBAHASAN

2.1 Penjelasan Objective Function

2.1.1 Objective Function

Dalam penyelesaian masalah optimisasi penempatan kendaraan, *objective function* berfungsi untuk mengukur kualitas setiap kemungkinan konfigurasi. Nilai yang dihasilkan memungkinkan algoritma membandingkan *state* awal dengan kandidat *state* berikutnya. Pada permasalahan ini, konfigurasi dengan nilai lebih besar diprioritaskan karena menunjukkan pendapatan pengiriman yang lebih tinggi.

Untuk *state* valid s, fungsi objektif didefinisikan sebagai $F(s) = \Sigma\ fee_i \times z_i$, dengan z_i bernilai 1 apabila kendaraan i diangkut dan 0 apabila berada di luar kapal. Pencarian melakukan maksimisasi. Satuan hasil adalah juta rupiah, mengikuti definisi tarif dalam spesifikasi.

Evaluator memeriksa kendaraan secara berurutan. Kendaraan di luar batas, bertumpuk dengan kendaraan yang telah dihitung, atau menyebabkan berat melampaui kapasitas tidak menyumbang tarif. Karena itu, pada *state* tidak valid, hasil bergantung pada urutan kendaraan. Eksperimen menghindari ambiguitas tersebut dengan menjaga *state* tetap valid dan memvalidasi kembali keluaran.

Tidak ada reward tambahan untuk kepadatan, banyak kendaraan, atau ETA. Metrik pemakaian ruang dan berat digunakan untuk menjelaskan hasil, bukan sebagai komponen optimasi. Dua layout berbeda dengan subset kendaraan sama dapat memiliki nilai objektif sama; keadaan ini menghasilkan *plateau* dalam ruang pencarian.

2.1.2 Source Code Objective Function

```
def calculate_objective(state: State) -> float:
    used_cells: set[tuple[int, int]] = set()
    total_weight = 0.0
    total_fee = 0.0

    for vehicle in state.vehicles:
        if not is_inside_ship(vehicle, state.ship):
            continue

        cells = occupied_cells(vehicle)
        if used_cells.intersection(cells):
            continue
        if total_weight + vehicle.weight > state.ship.max_capacity:
            continue

        used_cells.update(cells)
        total_weight += vehicle.weight
        total_fee += vehicle.shipping_fee

    return total_fee
```

2.1.3 Kesimpulan Pemilihan Objective Function

Fungsi objektif berupa penjumlahan *ShippingFee* dipilih karena secara langsung merepresentasikan tujuan persoalan. Validitas penempatan ditangani melalui pemeriksaan *constraint*, sehingga penambahan reward kepadatan atau penalti ETA tidak diperlukan pada model yang diuji. Konfigurasi yang memuat kendaraan dengan tarif total sama dapat memiliki nilai identik meskipun tata letaknya berbeda. Kondisi tersebut menjadi salah satu penyebab *plateau* pada proses pencarian.

2.2 Penjelasan Pemilihan Initial State

a. Source Code Random Initial State

```
def create_random_initial_state(problem: State, seed: int | None = None) ->
State:
    rng = random.Random(seed)
    vehicles = deepcopy(problem.vehicles)
    rng.shuffle(vehicles)

    state = State(ship=problem.ship, vehicles=vehicles)
    used_cells: set[tuple[int, int]] = set()
    total_weight = 0.0

    for vehicle in state.vehicles:
        vehicle.x = None
        vehicle.y = None
        vehicle.orientation = rng.choice(list(Orientation))

        if total_weight + vehicle.weight > state.ship.max_capacity:
            continue
```

```
        max_x = state.ship.width - vehicle.occupied_width
        max_y = state.ship.length - vehicle.occupied_length
        if max_x < 0 or max_y < 0:
            continue

        positions = [
            (x, y)
            for x in range(max_x + 1)
            for y in range(max_y + 1)
        ]
        rng.shuffle(positions)

        for x, y in positions:
            vehicle.x = x
            vehicle.y = y
            cells = occupied_cells(vehicle)

            if not used_cells.intersection(cells):
                used_cells.update(cells)
                total_weight += vehicle.weight
                break

        else:
            vehicle.x = None
            vehicle.y = None

    update_state_value(state)
    return state
```

b. Penjelasan Random Initial State

Urutan kendaraan, orientasi, dan daftar posisi diacak dengan generator lokal ber-seed. Kendaraan ditempatkan pada posisi pertama dalam daftar acak yang tidak menabrak muatan sebelumnya. Jika kapasitas tidak mencukupi, orientasi tidak muat, atau tidak ditemukan ruang, kendaraan tetap di luar. Strategi ini membentuk *state* awal valid tetapi tidak menjamin pendapatan atau kepadatan optimal.

Pengacakan dilakukan menggunakan seed agar percobaan dapat diulang. Pada setiap kendaraan, orientasi dipilih terlebih dahulu, kemudian kandidat koordinat diacak dan diperiksa. Posisi pertama yang tidak bertumpuk dengan kendaraan sebelumnya digunakan sebagai penempatan. Cara ini menjaga kelayakan konfigurasi awal, tetapi belum mempertimbangkan perbandingan tarif terhadap berat maupun luas kendaraan.

2.3 Penjelasan Penerapan Constraints

a. Source Code Penerapan Constraints

```
def constraint_violations(state: State) -> list[str]:
    violations = []

    if not unloaded_vehicles_are_outside(state):
        violations.append("invalid_outside_position")
    if not vehicles_are_inside_ship(state):
        violations.append("out_of_bounds")
    if not has_no_overlap(state):
        violations.append("overlap")
    if not is_within_capacity(state):
        violations.append("capacity_exceeded")

    return violations
```

```
def has_no_overlap(state: State) -> bool:
    used_cells: set[tuple[int, int]] = set()

    for vehicle in state.vehicles:
        if not vehicle.is_loaded:
            continue

        cells = occupied_cells(vehicle)
        if used_cells.intersection(cells):
            return False
        used_cells.update(cells)

    return True
```

b. Penjelasan Penerapan Constraints

Pemeriksaan overlap membentuk himpunan koordinat sel yang sudah dipakai. Jika sel kendaraan berikutnya beririsan dengan himpunan tersebut, konfigurasi ditolak. Dua kendaraan boleh bersentuhan pada sisi karena tidak ada sel grid yang digunakan bersama.

Constraint geometris memperhitungkan orientasi melalui occupied_width dan occupied_length. Kendaraan vertikal memakai panjang asli sebagai lebar efektif serta lebar asli sebagai panjang efektif. Validator terpisah dari objective sehingga sebuah nilai yang tampak tinggi tidak dipakai sebagai satu-satunya bukti kelayakan.

Pada 24 keluaran akhir eksperimen, validator harus menyatakan valid sebelum hasil dicatat. Selain itu, nilai akhir diperiksa agar tidak melampaui batas atas knapsack. Pengujian ini memeriksa dua jenis kegagalan yang berbeda: konfigurasi ilegal dan nilai tujuan yang tidak masuk akal.

c. Pembentukan Neighbor

```
def random_neighbor(
    state: State,
    rng: random.Random,
    max_attempts: int = 100,
) -> State:
    if not state.vehicles:
        raise ValueError("State has no vehicles")

    operations = ["move", "rotate"]
    if len(state.vehicles) >= 2:
        operations.append("swap")

    for _ in range(max_attempts):
        operation = rng.choice(operations)

        try:
            if operation == "move":
                vehicle = rng.choice(state.vehicles)
                if rng.random() < 0.2:
                    return move_vehicle(state, vehicle.id, None, None)

                max_x = state.ship.width - vehicle.occupied_width
                max_y = state.ship.length - vehicle.occupied_length
                if max_x >= 0 and max_y >= 0:
                    return move_vehicle(
                        state,
                        vehicle.id,
                        rng.randint(0, max_x),
                        rng.randint(0, max_y),
                    )

            elif operation == "swap":
                first, second = rng.sample(state.vehicles, 2)
                return swap_vehicles(state, first.id, second.id)

            else:
                vehicle = rng.choice(state.vehicles)
                return rotate_vehicle(state, vehicle.id)
        except ValueError:
            continue

    raise RuntimeError("Could not generate a valid neighbor")
```

Sampler mencoba paling banyak 100 kandidat. Operasi tidak valid ditolak dan dicoba ulang. RuntimeError berarti sampler gagal menemukan *neighbor* valid dalam batas percobaan, bukan bukti bahwa tidak ada satu pun *neighbor* valid. Move memiliki peluang 0,2 untuk meminta pembongkaran; peluang ini hanya berlaku setelah operator move terpilih.

2.4 Penjelasan Implementasi Algoritma Local Search

2.4.1 Stochastic Hill-Climbing

a. Deskripsi Stochastic Hill-Climbing

Stochastic Hill-Climbing merupakan varian hill-climbing yang mengevaluasi *neighbor* secara acak. Setiap iterasi membandingkan nilai kandidat dengan *state* saat ini. Kandidat diterima apabila memberikan peningkatan nilai, sedangkan kandidat dengan nilai sama atau lebih rendah ditolak.

b. Source Code Stochastic Hill-Climbing dalam Bahasa Pemrograman Python

```
def stochastic_hill_climbing(
    initial_state: State,
    max_iterations: int = 1000,
    max_no_improvement: int = 100,
    seed: int | None = None,
) -> HillClimbingResult:
    if max_iterations <= 0:
        raise ValueError("Maximum iterations must be positive")
    if max_no_improvement <= 0:
        raise ValueError("Maximum no improvement must be positive")

    rng = random.Random(seed)
    initial = deepcopy(initial_state)
    current = deepcopy(initial_state)
    update_state_value(initial)
    update_state_value(current)

    history = [current.value]
    no_improvement = 0
    iterations = 0

    for iteration in range(1, max_iterations + 1):
        try:
            neighbor = random_neighbor(current, rng)
        except RuntimeError:
            break

        if neighbor.value > current.value:
            current = neighbor
            no_improvement = 0
        else:
            no_improvement += 1

        history.append(current.value)
        iterations = iteration

        if no_improvement >= max_no_improvement:
            break

    return HillClimbingResult(
        initial_state=initial,
        final_state=current,
        objective_history=history,
        iterations=iterations,
    )
```

c. Penjelasan Source Code Stochastic Hill-Climbing

Setiap iterasi mengevaluasi satu *neighbor* valid acak dan hanya menerima peningkatan ketat. *State* bernilai sama ditolak. Proses berhenti setelah 100 percobaan tanpa perbaikan, setelah 1.000 iterasi, atau saat sampler *neighbor* gagal. Penghentian karena stagnasi bukan sertifikat local optimum sebab seluruh *neighbor*hood tidak dienumerasi.

2.4.2 Simulated Annealing

a. Deskripsi Simulated Annealing

Simulated Annealing merupakan algoritma pencarian lokal yang dapat menerima kandidat dengan nilai lebih rendah berdasarkan probabilitas tertentu. Pendekatan ini memungkinkan pencarian meninggalkan konfigurasi yang belum dapat diperbaiki melalui satu langkah peningkatan langsung. Probabilitas penerimaan dikendalikan oleh temperatur yang berkurang selama proses pencarian.

b. Source Code Simulated Annealing dalam Bahasa Pemrograman Python

```
def simulated_annealing(initial, seed, iterations=1000, temperature=100.,
cooling=.995):
    """Maximize fee, retain best state; plateau episodes are not proven
local optima."""
    rng = random.Random(seed)
    current = deepcopy(initial)
    best = deepcopy(current)
    history, best_history, probabilities, temperatures = [current.value],
[best.value], [], []
    accepted_worse = rejected_worse = plateau_episodes = stale = 0
    for i in range(iterations):
        T = temperature * cooling ** i
        try:
            neighbor = random_neighbor(current, rng)
        except RuntimeError:
            break
        delta = neighbor.value - current.value
        p = 1. if delta >= 0 else math.exp(delta / T)
        temperatures.append(T)
        probabilities.append(p)
        if delta >= 0 or rng.random() < p:
            accepted_worse += int(delta < 0)
            current = neighbor
        else:
            rejected_worse += int(delta < 0)
        if current.value > best.value:
            best = deepcopy(current)
            stale = 0
        else:
            stale += 1
            if stale == 100:
                plateau_episodes += 1
        history.append(current.value)
        best_history.append(best.value)
    return dict(initial=initial, final=best, terminal=current,
history=history,
                best_history=best_history, probabilities=probabilities,
                temperatures=temperatures, accepted_worse=accepted_worse,
                rejected_worse=rejected_worse,
plateau_episodes=plateau_episodes)
```

c. Penjelasan Source Code Simulated Annealing

Simulated Annealing memungkinkan perpindahan ke *state* dengan nilai lebih rendah untuk melewati hambatan pencarian. Dengan Δ = F(*neighbor*) − F(current), peningkatan dan perpindahan setara diterima. Penurunan diterima dengan probabilitas exp(Δ/T). Temperatur menggunakan T_k = 100 × 0,995^k untuk k mulai dari nol.

Program menyimpan *state* saat ini dan *state* terbaik secara terpisah. Solusi akhir yang dilaporkan adalah *state* terbaik selama pencarian; terminal *state* juga disimpan pada JSON. Dengan demikian, penurunan yang diterima tidak menghapus hasil terbaik yang telah ditemukan.

Untuk Δ negatif, probabilitas penerimaan berada di antara nol dan satu. Sebagai contoh, penurunan 20 pada T = 100 mempunyai probabilitas exp(−0,2) ≈ 0,819. Pada T = 5,

probabilitasnya turun menjadi exp(−4) ≈ 0,018. Suhu tinggi mendorong eksplorasi, sedangkan suhu rendah membatasi penerimaan penurunan.

Grafik probabilitas menampilkan min(1, exp(Δ/T)) untuk setiap kandidat valid yang dievaluasi. Nilai satu pada grafik dapat berasal dari kandidat lebih baik maupun kandidat setara, sehingga garis tersebut tidak boleh dibaca sebagai frekuensi penerimaan kandidat yang lebih buruk saja. Jumlah penerimaan penurunan dicatat secara terpisah.

Frekuensi stuck dioperasionalkan sebagai banyaknya episode yang mencapai 100 iterasi berturut-turut tanpa memperbaiki *best-so-far*. Setelah mencapai 100, satu episode dihitung satu kali dan tidak dihitung lagi sampai muncul rekor baru yang mereset penghitung. Definisi ini mengukur stagnasi, bukan frekuensi local optimum yang dibuktikan secara matematis.

Nilai temperatur bukan hasil tuning optimal. Kesimpulan perbandingan *Simulated Annealing* berlaku untuk jadwal ini dan benchmark yang digunakan. Pergantian distribusi tarif akan mengubah skala Δ sehingga dapat memerlukan temperatur berbeda.

2.4.3 Genetic Algorithm

a. Deskripsi Genetic Algorithm

Genetic Algorithm menggunakan sekumpulan kandidat solusi yang disebut populasi. Setiap individu merepresentasikan satu konfigurasi lengkap penempatan kendaraan. Populasi diperbarui melalui seleksi, *crossover*, mutasi, dan elitisme untuk mempertahankan kandidat berkualitas sekaligus membentuk konfigurasi baru.

b. Source Code Genetic Algorithm dalam Bahasa Pemrograman Python

```
def genetic_algorithm(
    problem: State,
    population_size: int = 50,
    generations: int = 100,
    mutation_rate: float = 0.1,
    elitism_count: int = 1,
    tournament_size: int = 3,
    seed: int | None = None,
) -> GeneticAlgorithmResult:
    if population_size < 2:
        raise ValueError("Population size must be at least 2")
    if generations <= 0:
        raise ValueError("Generations must be positive")
    if not 0 <= mutation_rate <= 1:
        raise ValueError("Mutation rate must be between 0 and 1")
    if not 0 <= elitism_count < population_size:
        raise ValueError("Elitism count must be smaller than population
size")
    if tournament_size <= 0:
```

```
        raise ValueError("Tournament size must be positive")

    rng = random.Random(seed)
    population = [
        create_random_initial_state(problem, rng.randrange(2**32))
        for _ in range(population_size)
    ]

    initial_best = deepcopy(max(population, key=lambda state: state.value))
    best = deepcopy(initial_best)
    maximum, average = _record_population(population)
    maximum_history = [maximum]
    average_history = [average]

    for _ in range(generations):
        population.sort(key=lambda state: state.value, reverse=True)
        next_population = [deepcopy(state) for state in
population[:elitism_count]]

        while len(next_population) < population_size:
            first = _tournament_selection(population, rng, tournament_size)
            second = _tournament_selection(population, rng, tournament_size)
            child = _crossover(first, second, rng)

            if rng.random() < mutation_rate:
                child = _mutate(child, rng)

            next_population.append(child)

        population = next_population
        generation_best = max(population, key=lambda state: state.value)
        if generation_best.value > best.value:
            best = deepcopy(generation_best)

        maximum, average = _record_population(population)
        maximum_history.append(maximum)
        average_history.append(average)

    return GeneticAlgorithmResult(
        initial_best_state=initial_best,
        best_state=best,
        maximum_history=maximum_history,
        average_history=average_history,
        generations=generations,
        population_size=population_size,
    )
```

```
def _tournament_selection(
    population: list[State],
    rng: random.Random,
    tournament_size: int,
) -> State:
    candidates = rng.sample(population, min(tournament_size,
len(population)))
    return max(candidates, key=lambda state: state.value)
```

```
def _crossover(first: State, second: State, rng: random.Random) -> State:
    child = deepcopy(first)
    second_vehicles = {vehicle.id: vehicle for vehicle in second.vehicles}

    for vehicle in child.vehicles:
        source = rng.choice((vehicle, second_vehicles[vehicle.id]))
        vehicle.x = source.x
        vehicle.y = source.y
        vehicle.orientation = source.orientation

    return _repair_state(child)
```

```
def _repair_state(state: State) -> State:
    candidate = deepcopy(state)
    placements = {
        vehicle.id: (vehicle.x, vehicle.y, vehicle.orientation)
        for vehicle in candidate.vehicles
    }

    for vehicle in candidate.vehicles:
        vehicle.x = None
        vehicle.y = None

    for vehicle in candidate.vehicles:
        x, y, orientation = placements[vehicle.id]
        vehicle.orientation = orientation
        vehicle.x = x
        vehicle.y = y

        if not is_valid_state(candidate):
            vehicle.x = None
            vehicle.y = None

    update_state_value(candidate)
    return candidate
```

```
def _mutate(state: State, rng: random.Random) -> State:
    try:
        return random_neighbor(state, rng)
    except (RuntimeError, ValueError):
        return deepcopy(state)
```

c. Penjelasan Source Code Genetic Algorithm

Satu individu adalah satu *State* lengkap. Populasi awal terdiri atas *state* acak valid yang memakai seed turunan dari generator utama. Berbeda dari Hill Climbing dan *Simulated Annealing*, GA memulai pencarian dari beberapa kandidat. *State* awal yang divisualisasikan adalah individu terbaik pada populasi generasi nol, bukan satu-satunya individu awal.

Tournament memilih tiga kandidat tanpa pengembalian, kemudian mengambil kandidat bernilai tertinggi. Mekanisme ini memberi tekanan seleksi pada individu baik, tetapi tetap

memungkinkan individu lain terpilih bergantung pada sampel turnamen. Ukuran turnamen dikendalikan tetap pada seluruh eksperimen.

Untuk setiap kendaraan, posisi dan orientasinya diwariskan dari salah satu parent secara acak. Kendaraan dicocokkan berdasarkan ID sehingga urutan list parent tidak harus sama. Pewarisan posisi secara independen dapat menimbulkan overlap atau kelebihan berat, sehingga hasil *crossover* wajib melewati *repair*.

Repair mengosongkan seluruh koordinat lalu mencoba memulihkan penempatan kendaraan satu per satu. Jika pemulihan melanggar *constraint*, kendaraan tersebut tetap di luar. Hasil akhirnya valid, tetapi keputusan *repair* dipengaruhi urutan list kendaraan; kendaraan yang diproses awal mendapatkan prioritas ruang dan berat.

Mutasi memakai satu *neighbor* valid acak. Probabilitas mutasi 0,1 berlaku per anak, bukan per kendaraan. Apabila mutasi gagal menghasilkan kandidat, *state* disalin tanpa perubahan. Elitisme sebesar satu menyimpan individu terbaik generasi sebelumnya, sehingga nilai maksimum populasi tidak menurun dalam pengaturan yang digunakan.

Setiap generasi mengurutkan populasi, mempertahankan satu elite, lalu membentuk anak melalui tournament, *crossover*, *repair*, dan mutasi sampai ukuran populasi pulih. Program mencatat maksimum dan rata-rata populasi setelah setiap generasi. History juga memuat generasi nol; eksperimen 60 generasi karena itu memiliki 61 titik grafik.

Nilai maksimum merupakan kualitas kandidat terbaik, sedangkan rata-rata menggambarkan kualitas populasi secara keseluruhan. Rata-rata tinggi tidak otomatis membuktikan keragaman populasi baik. Populasi yang sangat mirip dapat memiliki rata-rata tinggi tetapi tidak lagi menghasilkan perbaikan. Dalam eksperimen, interpretasi stagnasi didasarkan pada history yang benar-benar terekam.

2.5 Hasil Eksperimen dan Analisis

Konfigurasi GA G=60 dan P=30 muncul pada kedua rangkaian. Keduanya tetap dieksekusi sebagai run terpisah agar desain tiga variasi pada setiap rangkaian lengkap. Dengan seed dan parameter sama, nilai serta history semestinya identik; waktu dapat berbeda. Kedua label ini tidak diperlakukan sebagai enam pengulangan independen.

HC dan SA memakai *state* awal yang identik untuk seed yang sama. GA menerima problem yang sama, tetapi membentuk populasi awal sendiri. Kesetaraan seed tidak membuat

awal GA identik dengan awal HC atau SA. Perbandingan ini menilai konfigurasi praktis, bukan eksperimen dengan jumlah evaluasi objective yang disamakan.

Durasi menggunakan perf_counter hanya di sekitar panggilan algoritma. Untuk HC dan SA, pembentukan satu *state* awal dilakukan sebelum timer; untuk GA, pembentukan populasi awal berlangsung di dalam fungsi dan termasuk timer. Pembuatan grafik, penulisan laporan, serta serialisasi hasil tidak termasuk durasi pencarian.

Windows-11-10.0.26200-SP0

3.12.14 (main, Aug 25 2026, 14:01:42) [MSC v.1944 64 bit (AMD64)]

Eksekusi dilakukan pada 21 Oktober 2026. Tiga seed memberikan gambaran variasi awal, tetapi belum cukup untuk inferensi statistik yang kuat atau generalisasi ke seluruh ukuran instance.

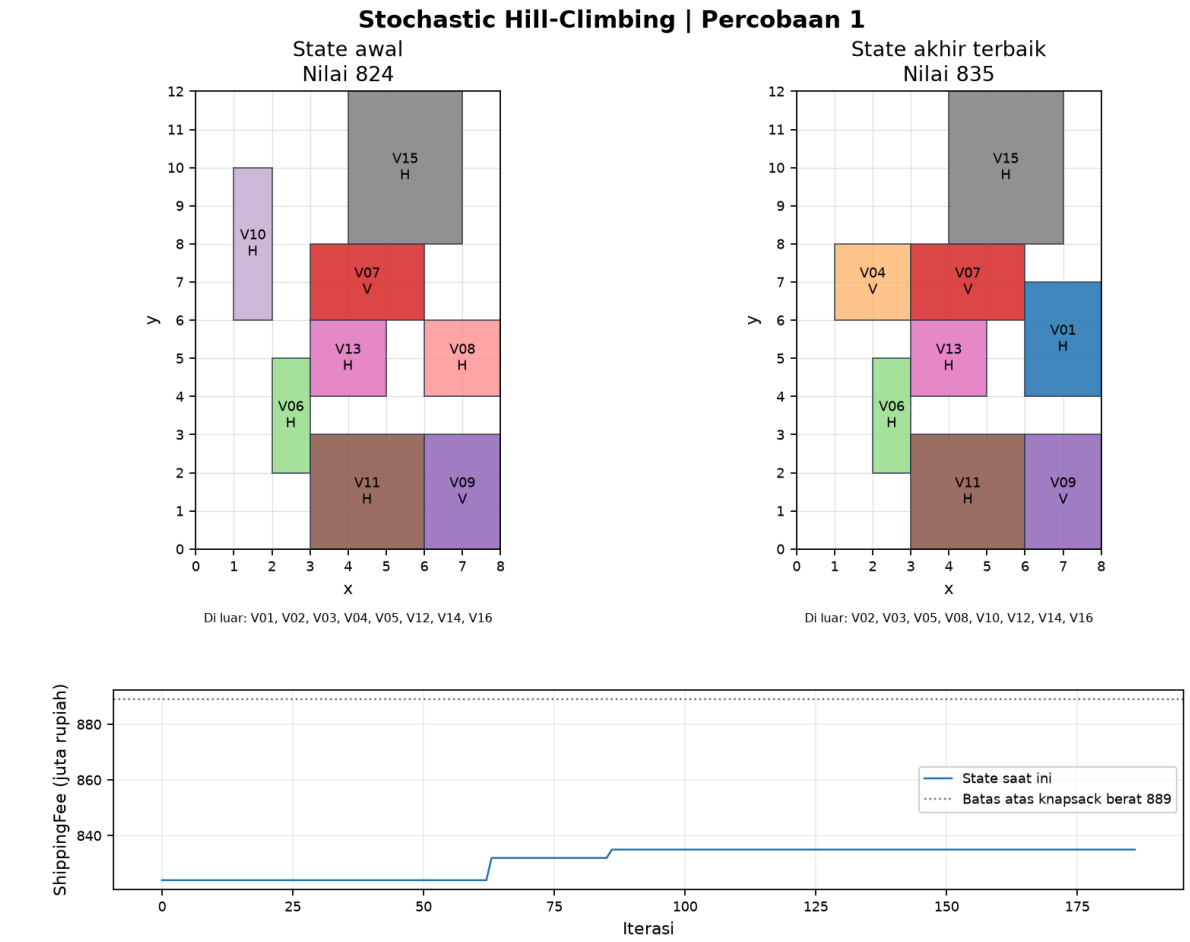
Tabel 2.1 Konfigurasi Eksperimen

Algoritma	**Parameter**	**Pengulangan**
Stochastic HC	Maksimum 1.000 iterasi; stagnasi 100	3
Simulated Annealing	1.000 iterasi; T0=100; alpha=0,995	3
GA variasi generasi	P=30; G=20,60,120; mutasi=0,1	3 tiap konfigurasi
GA variasi populasi	G=60; P=10,30,60; mutasi=0,1	3 tiap konfigurasi

Data uji terdiri atas 16 kendaraan pada geladak 8 × 12 dengan kapasitas berat 55. Data merupakan benchmark terkontrol untuk pengujian program. Seluruh rincian kendaraan disajikan pada Lampiran A, sedangkan koordinat awal dan akhir setiap percobaan tersedia pada Lampiran B. Nilai 889 digunakan sebagai batas atas berdasarkan relaksasi knapsack berat.

2.5.1 Stochastic Hill-Climbing

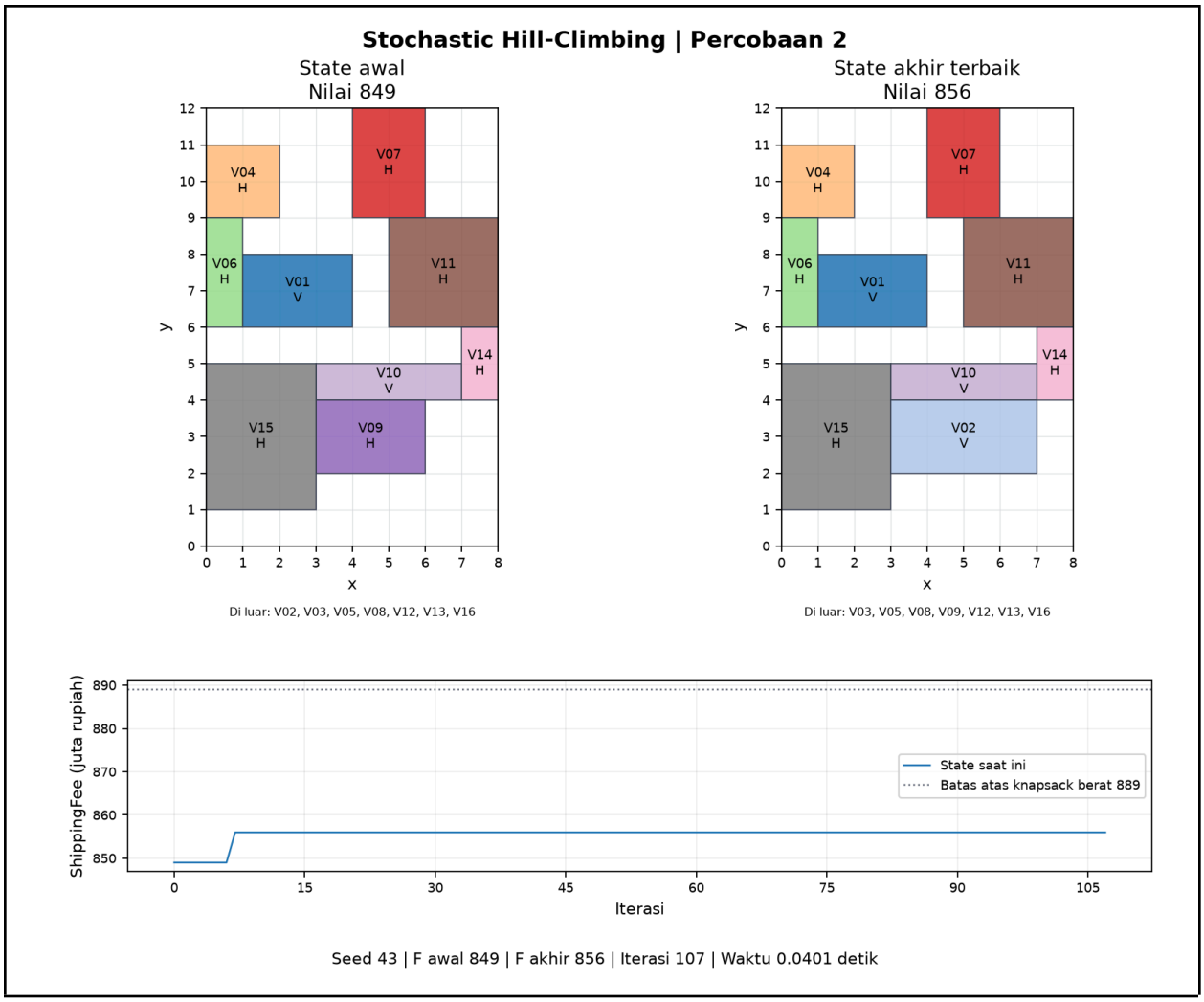
Percobaan Pertama



Gambar 2.1 Stochastic Hill-Climbing HC Percobaan Pertama

Pada percobaan dengan seed 42, nilai *objective function* berubah dari 824 menjadi 835. Pencarian berlangsung selama 0.0630 detik dengan 186 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

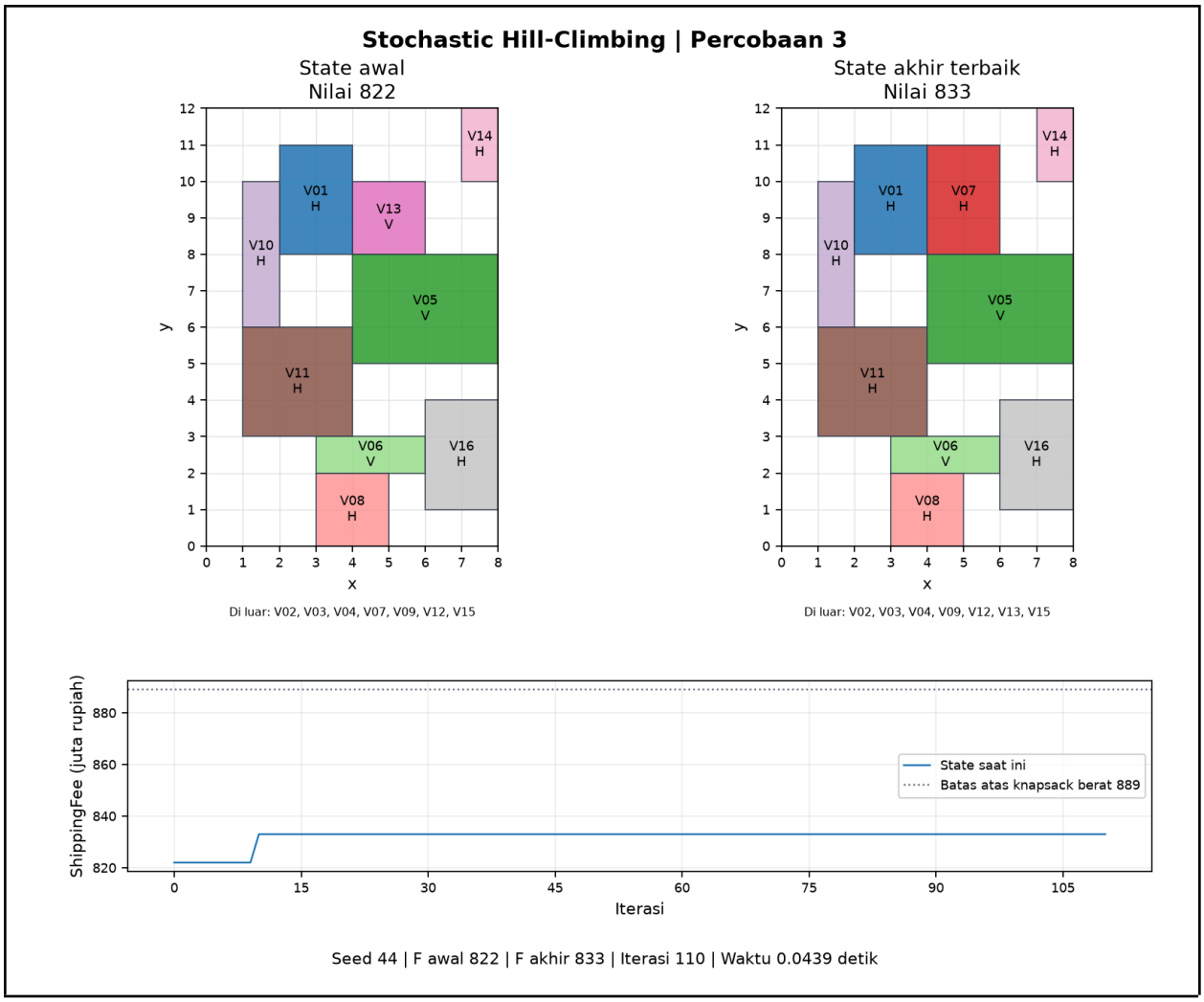
Percobaan Kedua



Gambar 2.2 Stochastic Hill-Climbing HC Percobaan Kedua

Pada percobaan dengan seed 43, nilai *objective function* berubah dari 849 menjadi 856. Pencarian berlangsung selama 0.0401 detik dengan 107 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

Percobaan Ketiga



Gambar 2.3 Stochastic Hill-Climbing HC Percobaan Ketiga

Pada percobaan dengan seed 44, nilai *objective function* berubah dari 822 menjadi 833. Pencarian berlangsung selama 0.0439 detik dengan 110 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

a. Analisis Hasil Percobaan Stochastic Hill-Climbing HC

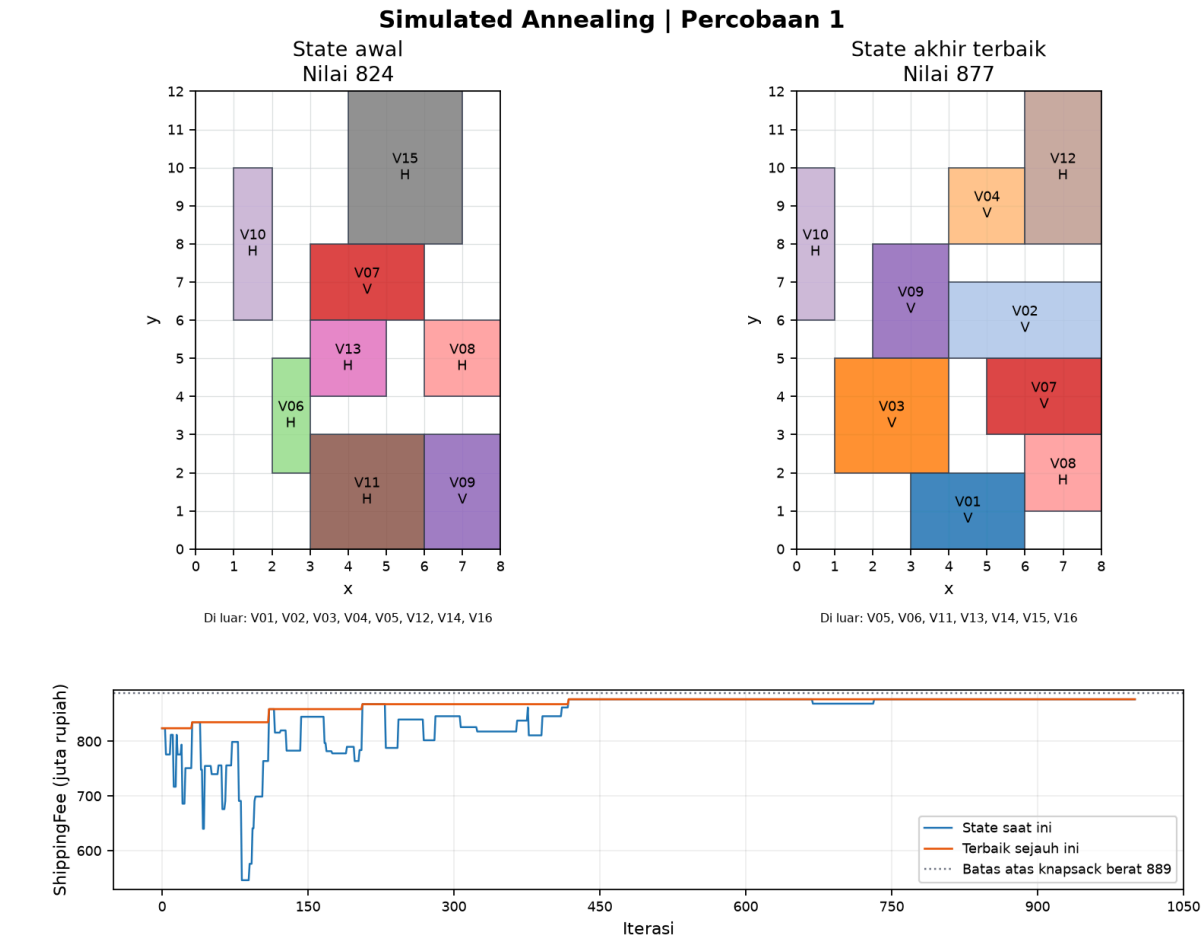
Dari ketiga percobaan, diperoleh rata-rata nilai akhir sebesar 841.33 juta rupiah, dengan nilai terendah 833 dan nilai tertinggi 856. Simpangan baku sampel sebesar 12.74 menunjukkan variasi nilai akhir antar-seed. Berdasarkan durasi 0.0630, 0.0401, dan 0.0439 detik, rata-rata durasi pencarian adalah 0.0490 detik.

Stochastic Hill-Climbing meningkatkan nilai awal masing-masing sebesar 11, 7, dan 11 juta rupiah, atau rata-rata 9.67 juta rupiah. Pencarian selesai setelah 186, 107, dan 110 iterasi,

sebelum batas maksimum 1.000 iterasi. Algoritma hanya menerima peningkatan nilai secara ketat sehingga perpindahan bernilai setara yang mungkin membantu penataan ulang tidak diterima. Pencarian berhenti setelah 100 iterasi tanpa peningkatan, tetapi kondisi ini tidak membuktikan bahwa solusi telah mencapai *local optimum* karena hanya sebagian tetangga yang diperiksa.

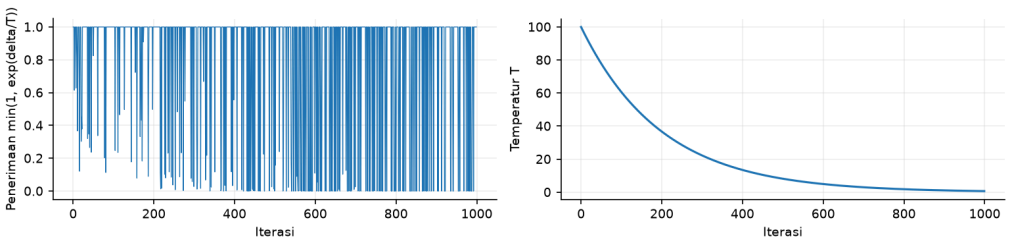
2.5.2 Simulated Annealing

Percobaan Pertama



Gambar 2.4 Simulated Annealing SA Percobaan Pertama

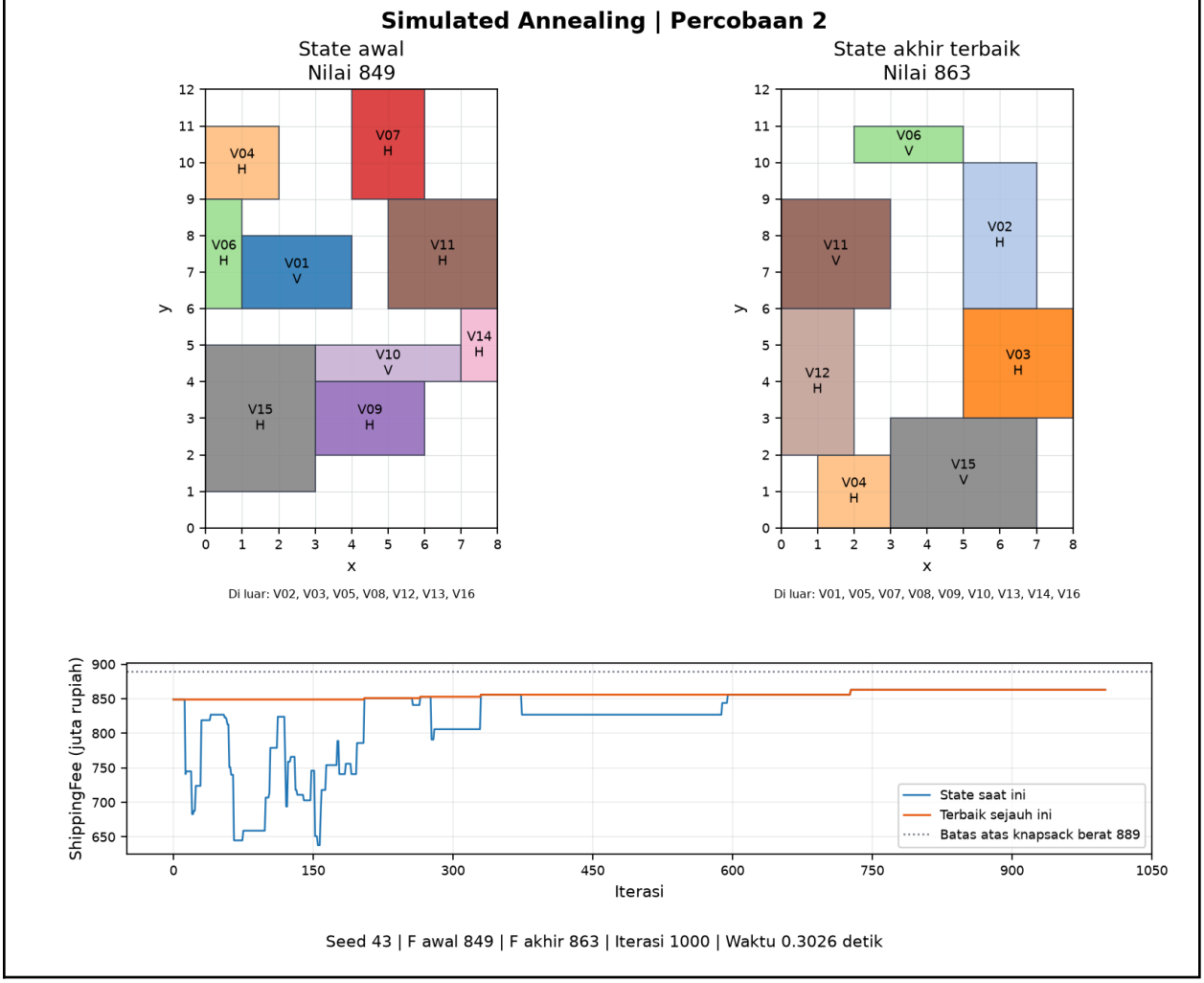
Pada percobaan dengan seed 42, nilai *objective function* berubah dari 824 menjadi 877. Pencarian berlangsung selama 0.3634 detik dengan 1000 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.



Gambar 2.5 Probabilitas Penerimaan dan Temperatur SA Seed 42

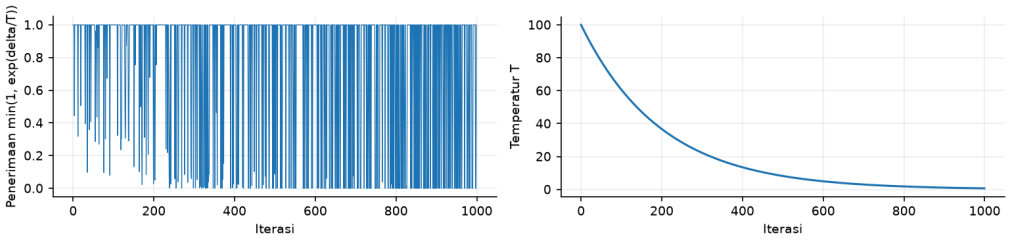
Jumlah kandidat dengan nilai lebih rendah yang diterima adalah 22, sedangkan 209 kandidat ditolak. Terdapat 2 episode stagnasi berdasarkan 100 iterasi tanpa rekor baru. Nilai akhir yang dilaporkan merupakan *best-so-far*; terminal *state* bernilai 877.

Percobaan Kedua



Gambar 2.6 Simulated Annealing SA Percobaan Kedua

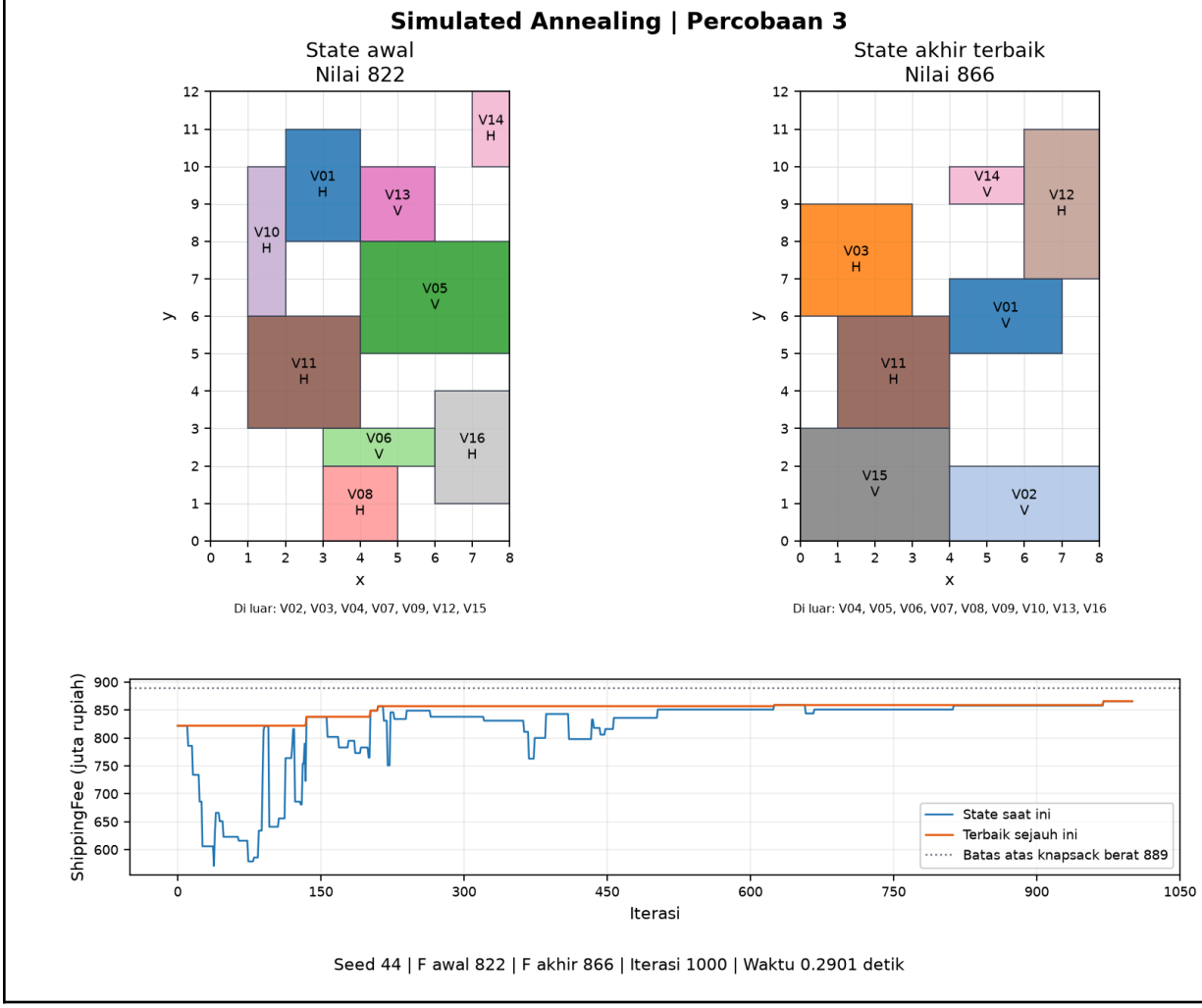
Pada percobaan dengan seed 43, nilai *objective function* berubah dari 849 menjadi 863. Pencarian berlangsung selama 0.3026 detik dengan 1000 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.



Gambar 2.7 Probabilitas Penerimaan dan Temperatur SA Seed 43

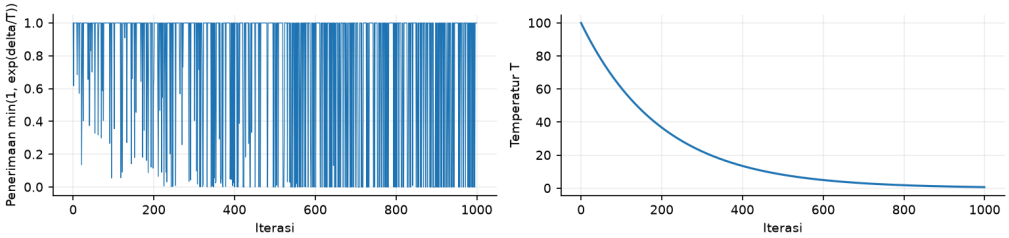
Jumlah kandidat dengan nilai lebih rendah yang diterima adalah 20, sedangkan 248 kandidat ditolak. Terdapat 3 episode stagnasi berdasarkan 100 iterasi tanpa rekor baru. Nilai akhir yang dilaporkan merupakan *best-so-far*; terminal *state* bernilai 863.

Percobaan Ketiga



Gambar 2.8 Simulated Annealing SA Percobaan Ketiga

Pada percobaan dengan seed 44, nilai *objective function* berubah dari 822 menjadi 866. Pencarian berlangsung selama 0.2901 detik dengan 1000 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.



Gambar 2.9 Probabilitas Penerimaan dan Temperatur SA Seed 44

Jumlah kandidat dengan nilai lebih rendah yang diterima adalah 28, sedangkan 242 kandidat ditolak. Terdapat 3 episode stagnasi berdasarkan 100 iterasi tanpa rekor baru. Nilai akhir yang dilaporkan merupakan *best-so-far*; terminal *state* bernilai 866.

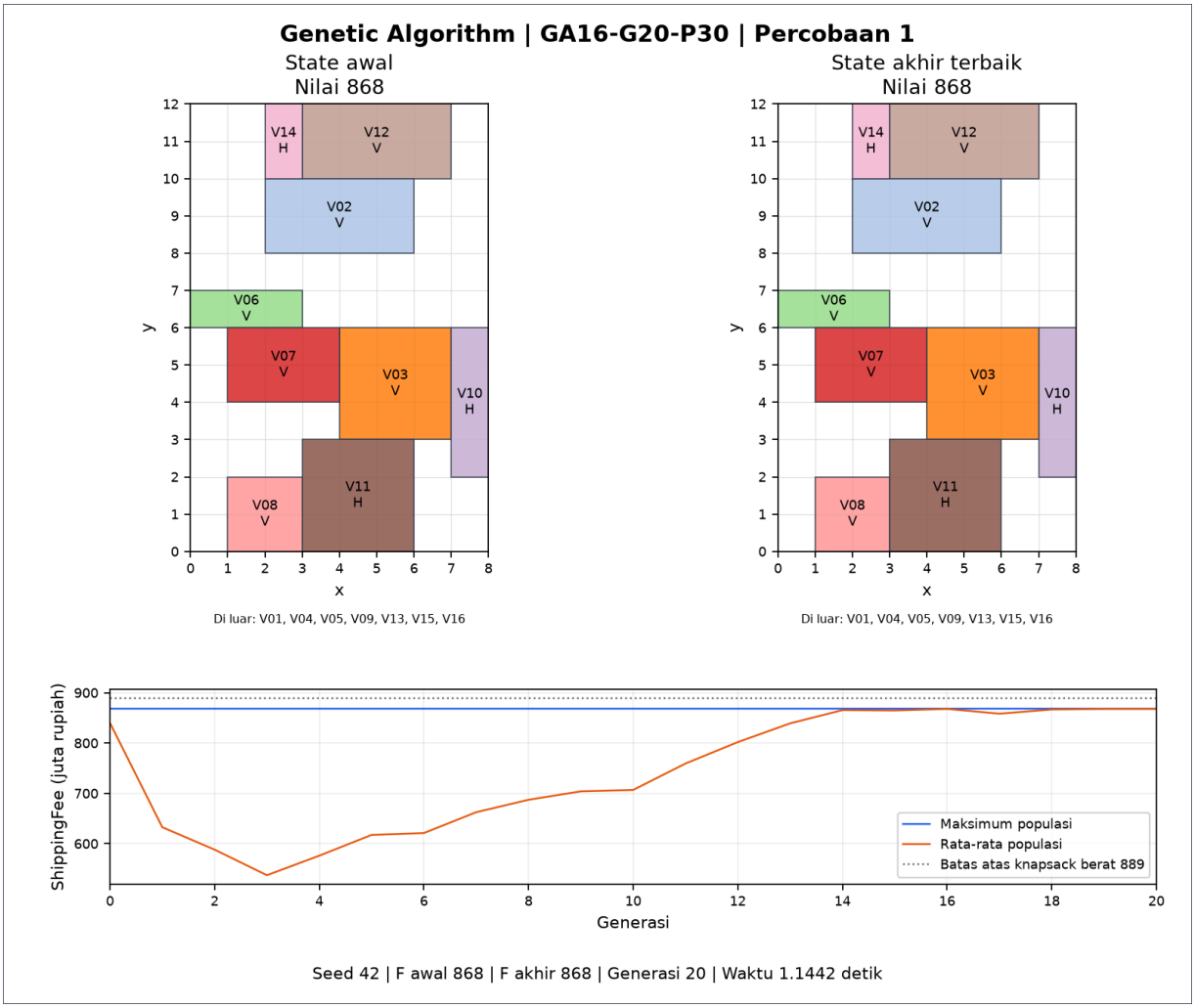
a. Analisis Hasil Percobaan Simulated Annealing SA

Dari ketiga percobaan, diperoleh rata-rata nilai akhir sebesar 868.67 juta rupiah, dengan nilai terendah 863 dan nilai tertinggi 877. Simpangan baku sampel sebesar 7.37 menunjukkan variasi nilai akhir antar-seed. Berdasarkan durasi 0.3634, 0.3026, dan 0.2901 detik, rata-rata durasi pencarian adalah 0.3187 detik.

Simulated Annealing meningkatkan nilai awal masing-masing sebesar 53, 14, dan 44 juta rupiah, atau rata-rata 37.00 juta rupiah, dalam 1.000 iterasi pada setiap percobaan. Algoritma menerima 22, 20, dan 28 kandidat dengan nilai lebih rendah sehingga pencarian dapat mengubah komposisi dan tata letak muatan. Seiring penurunan temperatur, peluang menerima penurunan dengan besar yang sama semakin kecil. Nilai akhir yang dilaporkan merupakan *best-so-far* dan pada ketiga percobaan sama dengan nilai *terminal state*. Jumlah episode stagnasi adalah 2, 3, dan 3 berdasarkan 100 iterasi tanpa rekor baru; ukuran ini tidak membuktikan *local optimum*.

2.5.3 Genetic Algorithm

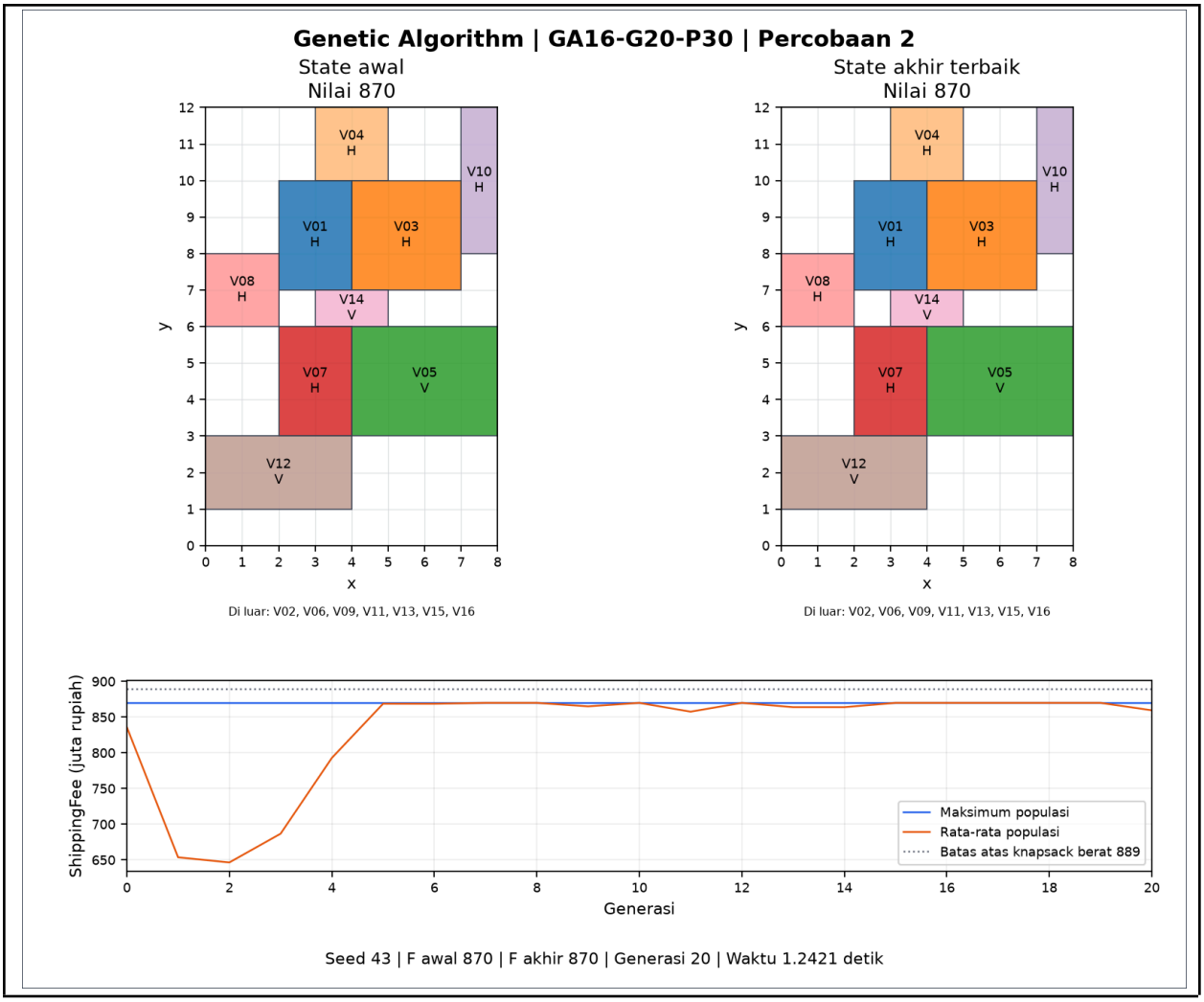
Variasi Banyak Generasi GA_G20

Percobaan Pertama


Gambar 2.10 Genetic Algorithm GA_G20 Percobaan Pertama

Pada percobaan dengan seed 42, nilai *objective function* berubah dari 868 menjadi 868. Pencarian berlangsung selama 1.1442 detik dengan 20 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

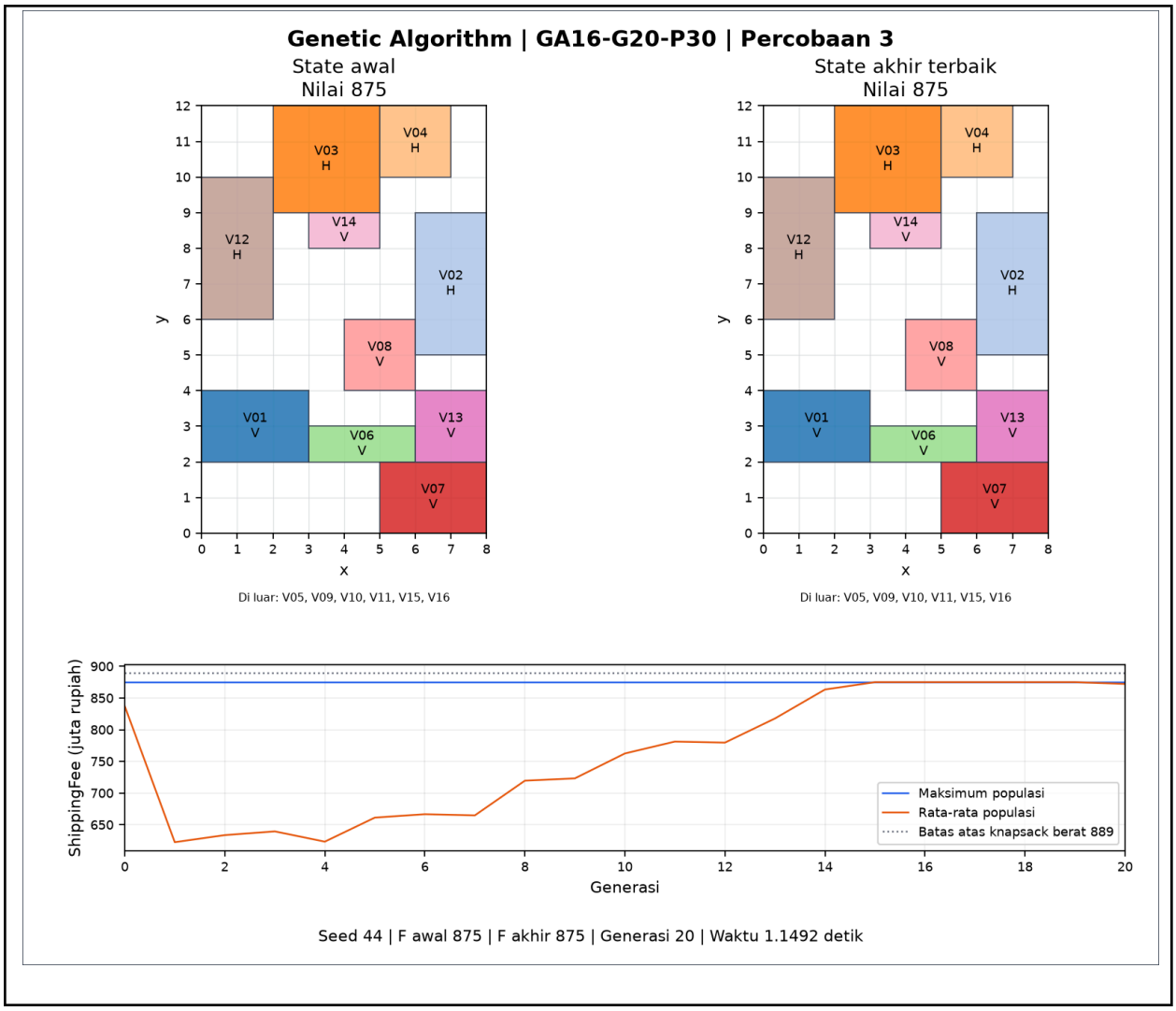
Percobaan Kedua



Gambar 2.11 Genetic Algorithm GA_G20 Percobaan Kedua

Pada percobaan dengan seed 43, nilai *objective function* berubah dari 870 menjadi 870. Pencarian berlangsung selama 1.2421 detik dengan 20 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

Percobaan Ketiga



Gambar 2.12 Genetic Algorithm GA_G20 Percobaan Ketiga

Pada percobaan dengan seed 44, nilai *objective function* berubah dari 875 menjadi 875. Pencarian berlangsung selama 1.492 detik dengan 20 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

a. Analisis Hasil Percobaan Genetic Algorithm GA_G20

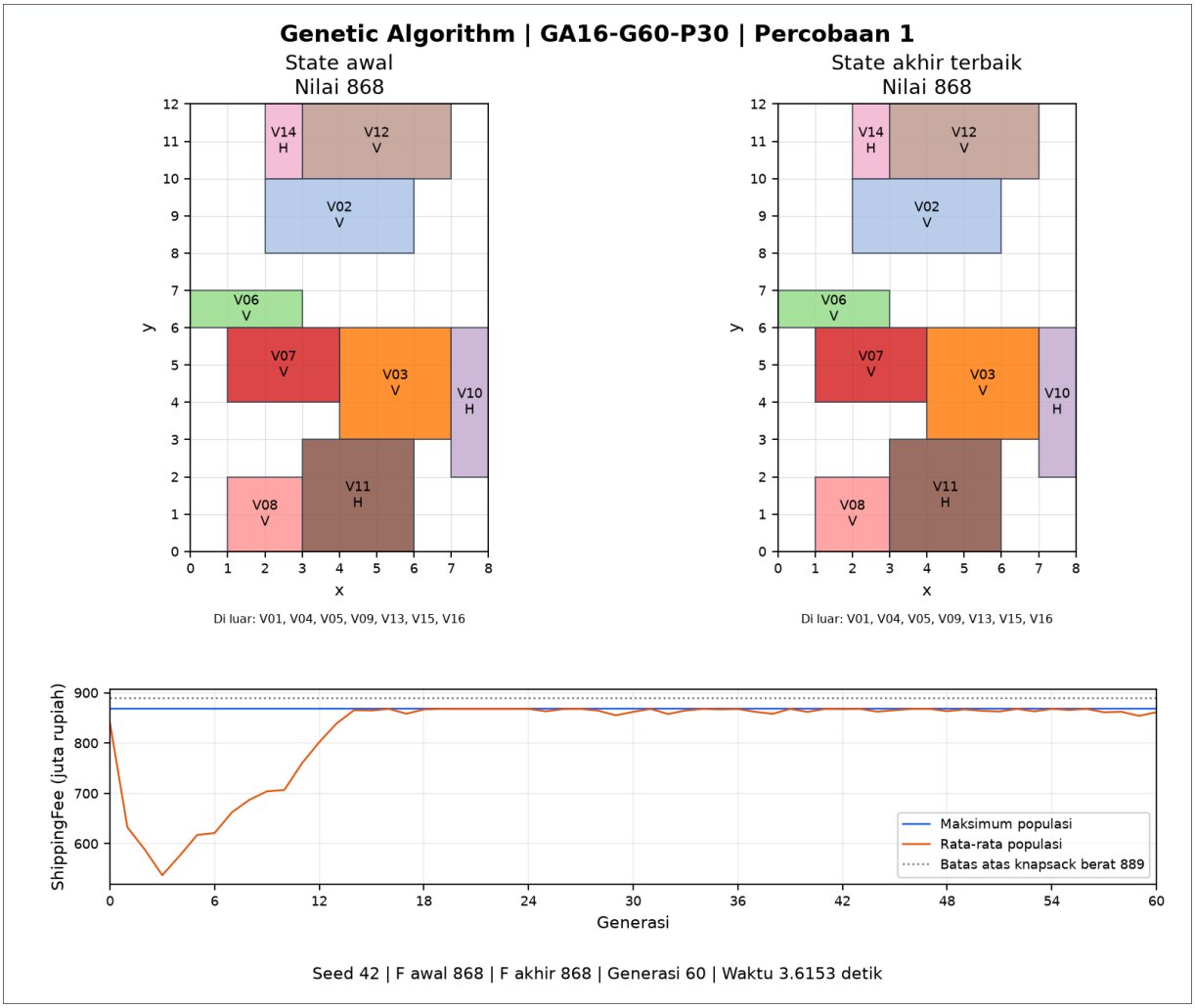
Dari ketiga percobaan, diperoleh rata-rata nilai akhir sebesar 871.00 juta rupiah, dengan nilai terendah 868 dan nilai tertinggi 875. Simpangan baku sampel sebesar 3.61 menunjukkan variasi kualitas akibat pengacakan. Rata-rata durasi pencarian adalah 1.2928 detik.

Elitisme menjaga nilai maksimum populasi agar tidak menurun, sedangkan rata-rata populasi dapat berubah akibat *crossover*, *repair*, dan mutasi. Kemiripan nilai antarindividu tidak

langsung membuktikan kesamaan susunan kendaraan. Penafsiran konvergensi perlu memperhatikan grafik maksimum serta rata-rata secara bersamaan.

Variasi Banyak Generasi GA_G60

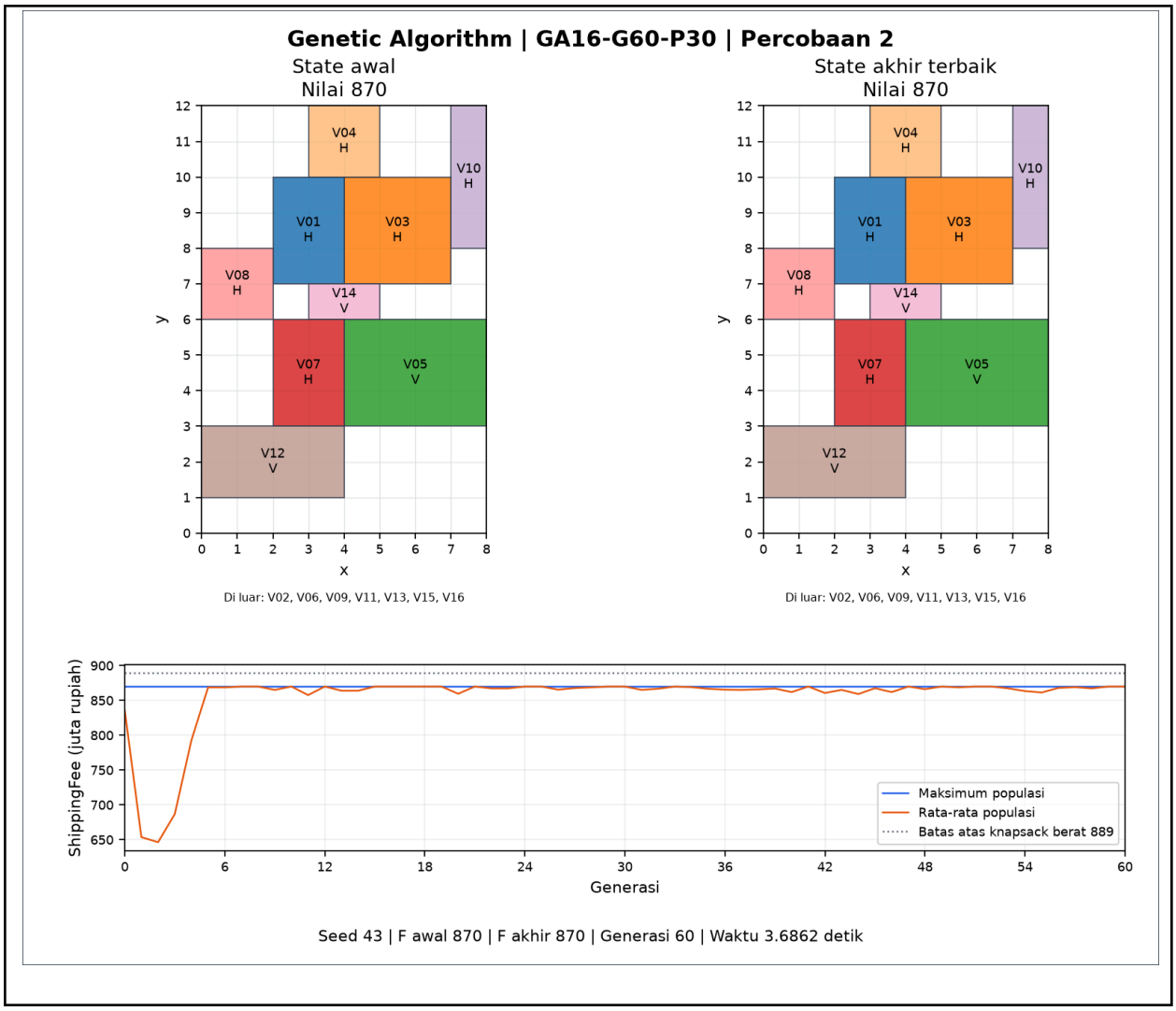
Percobaan Pertama



Gambar 2.13 Genetic Algorithm GA_G60 Percobaan Pertama

Pada percobaan dengan seed 42, nilai *objective function* berubah dari 868 menjadi 868. Pencarian berlangsung selama 3.6153 detik dengan 60 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

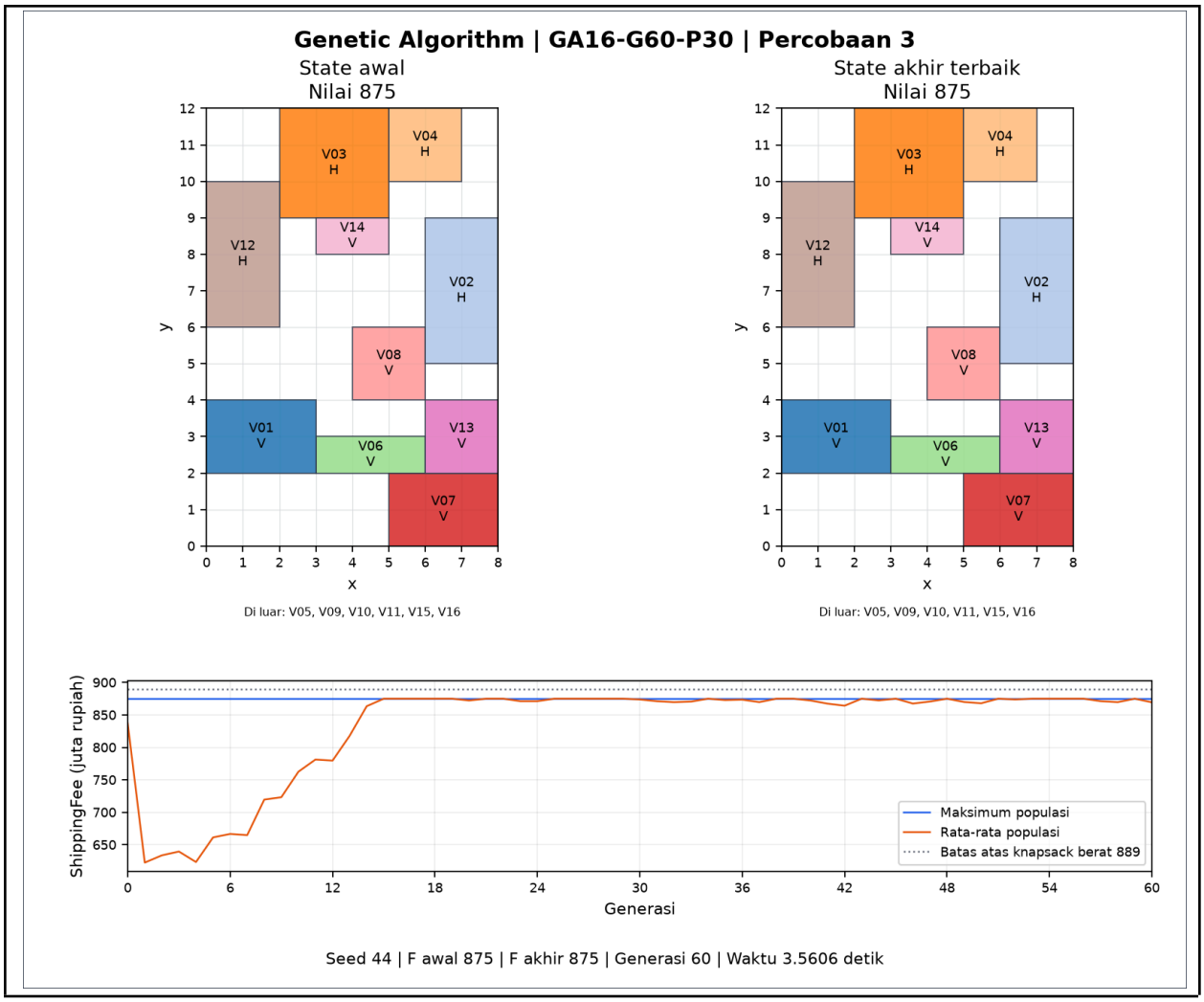
Percobaan Kedua



Gambar 2.14 Genetic Algorithm GA_G60 Percobaan Kedua

Pada percobaan dengan seed 43, nilai *objective function* berubah dari 870 menjadi 870. Pencarian berlangsung selama 3.6862 detik dengan 60 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

Percobaan Ketiga



Gambar 2.15 Genetic Algorithm GA_G60 Percobaan Ketiga

Pada percobaan dengan seed 44, nilai *objective function* berubah dari 875 menjadi 875. Pencarian berlangsung selama 3.5605 detik dengan 60 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

a. Analisis Hasil Percobaan Genetic Algorithm GA_G60

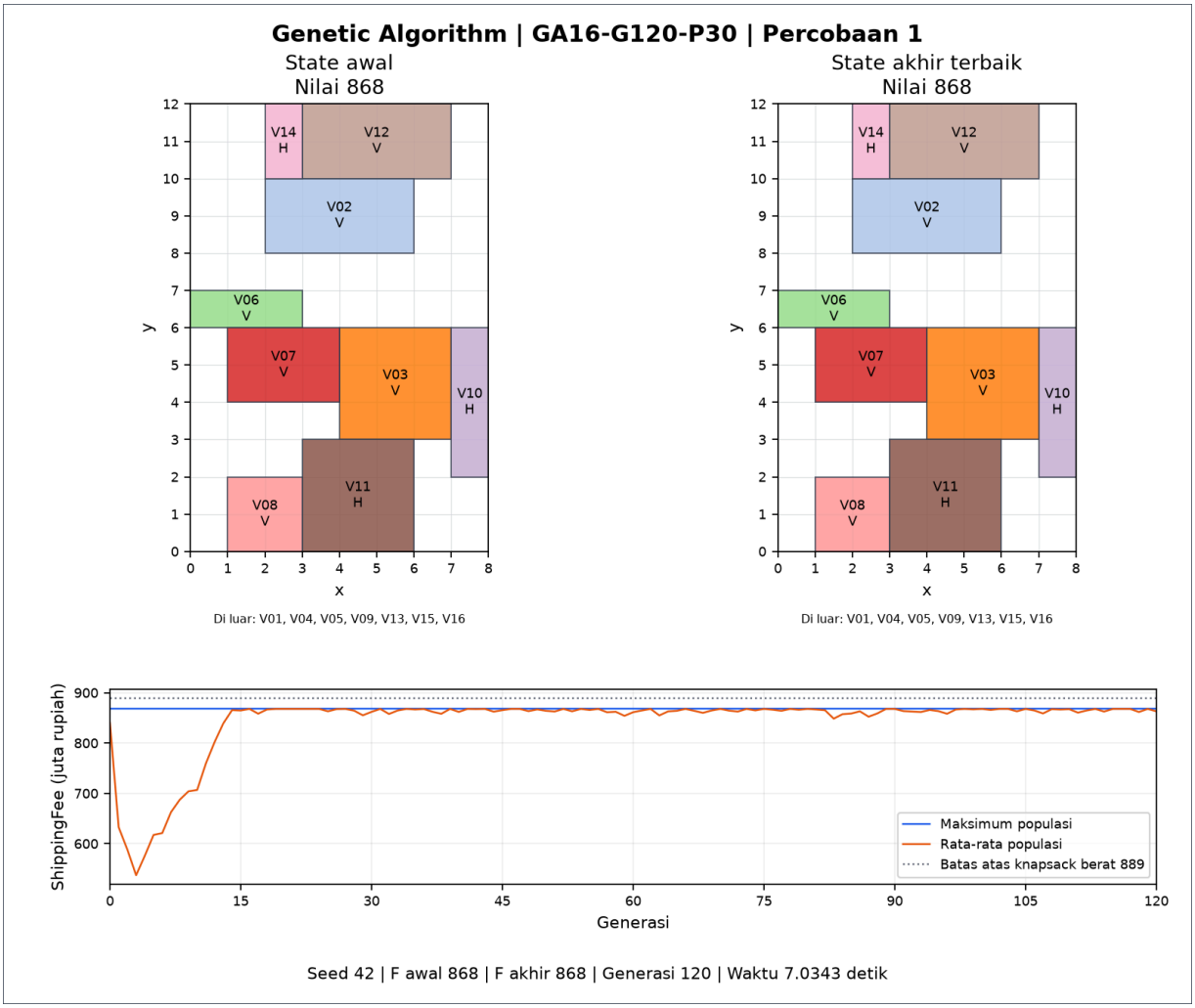
Dari ketiga percobaan, diperoleh rata-rata nilai akhir sebesar 871,00 juta rupiah, dengan nilai terendah 868 dan nilai tertinggi 875. Simpangan baku sampel sebesar 3,61 menunjukkan variasi kualitas akibat pengacakan. Rata-rata durasi pencarian adalah 3,6207 detik.

Elitisme menjaga nilai maksimum populasi agar tidak menurun, sedangkan rata-rata populasi dapat berubah akibat crossover, repair, dan mutasi. Kemiripan nilai antarindividu tidak

langsung membuktikan kesamaan susunan kendaraan. Penafsiran konvergensi perlu memperhatikan grafik maksimum serta rata-rata secara bersamaan.

Variasi Banyak Generasi GA_G120

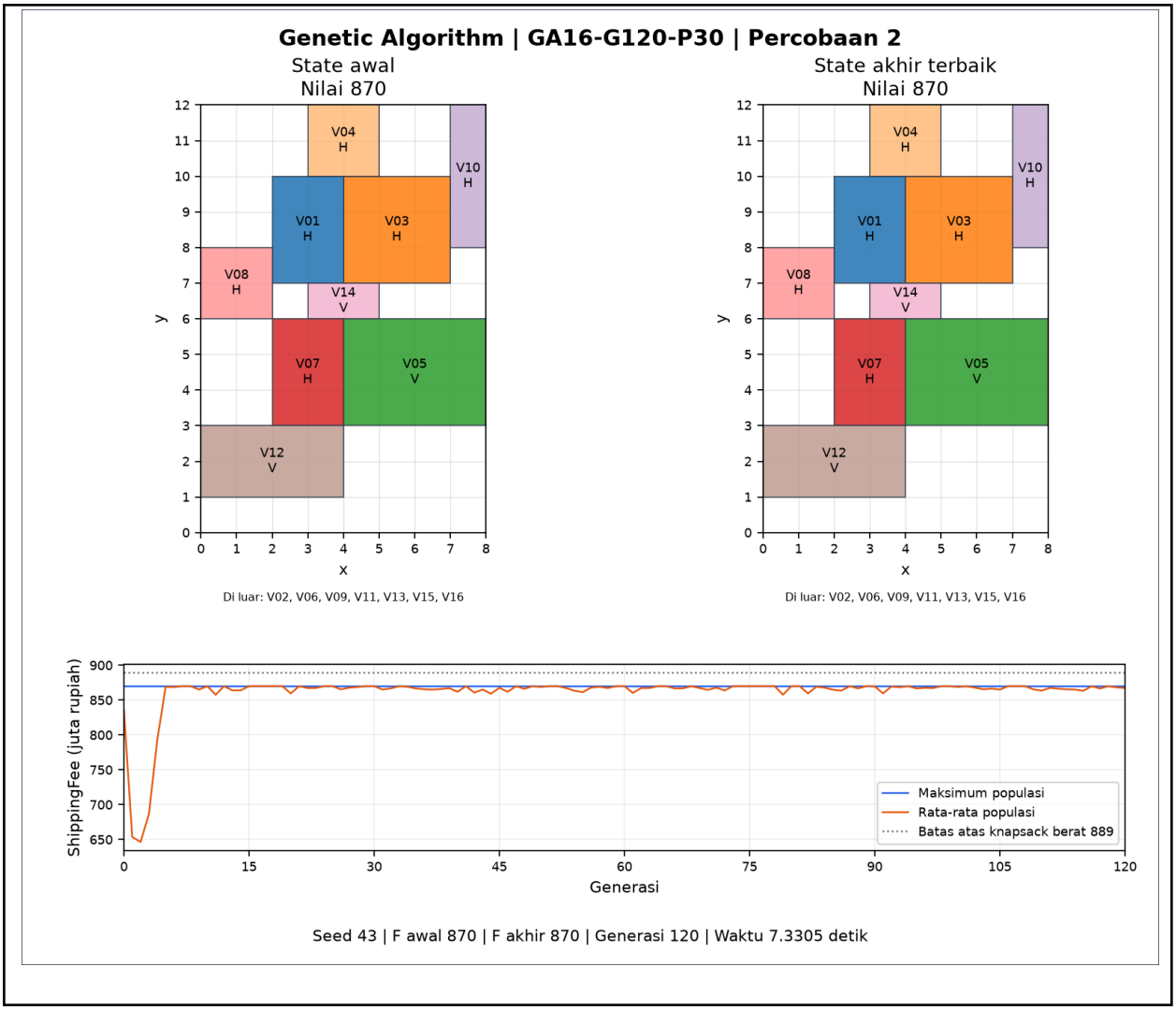
Percobaan Pertama



Gambar 2.16 Genetic Algorithm GA_G120 Percobaan Pertama

Pada percobaan dengan seed 42, nilai *objective function* berubah dari 868 menjadi 868. Pencarian berlangsung selama 7.0343 detik dengan 120 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

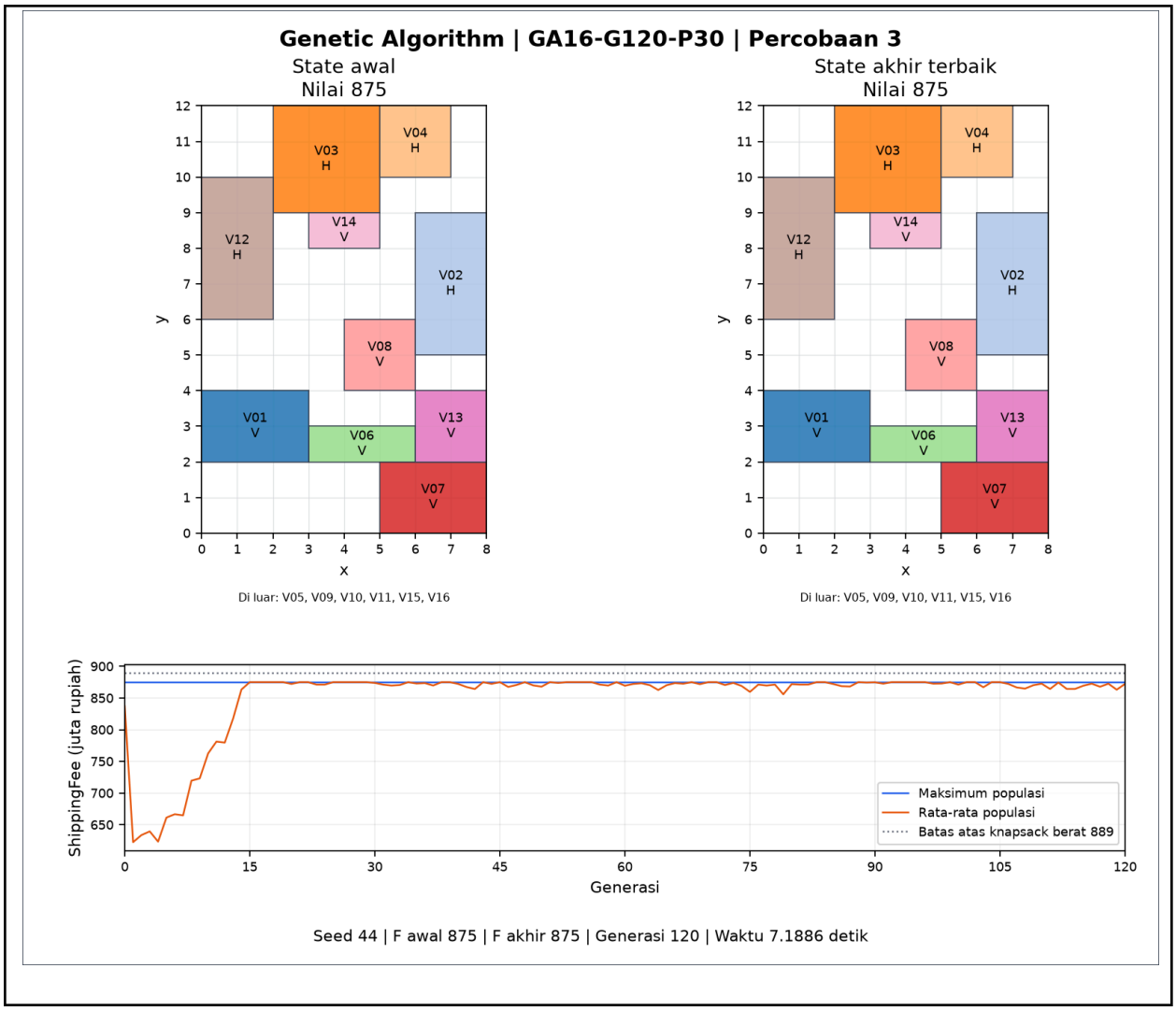
Percobaan Kedua



Gambar 2.17 Genetic Algorithm GA_G120 Percobaan Kedua

Pada percobaan dengan seed 43, nilai *objective function* berubah dari 870 menjadi 870. Pencarian berlangsung selama 7.3305 detik dengan 120 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

Percobaan Ketiga



Gambar 2.18 Genetic Algorithm GA_G120 Percobaan Ketiga

Pada percobaan dengan seed 44, nilai *objective function* berubah dari 875 menjadi 875. Pencarian berlangsung selama 7.1886 detik dengan 120 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

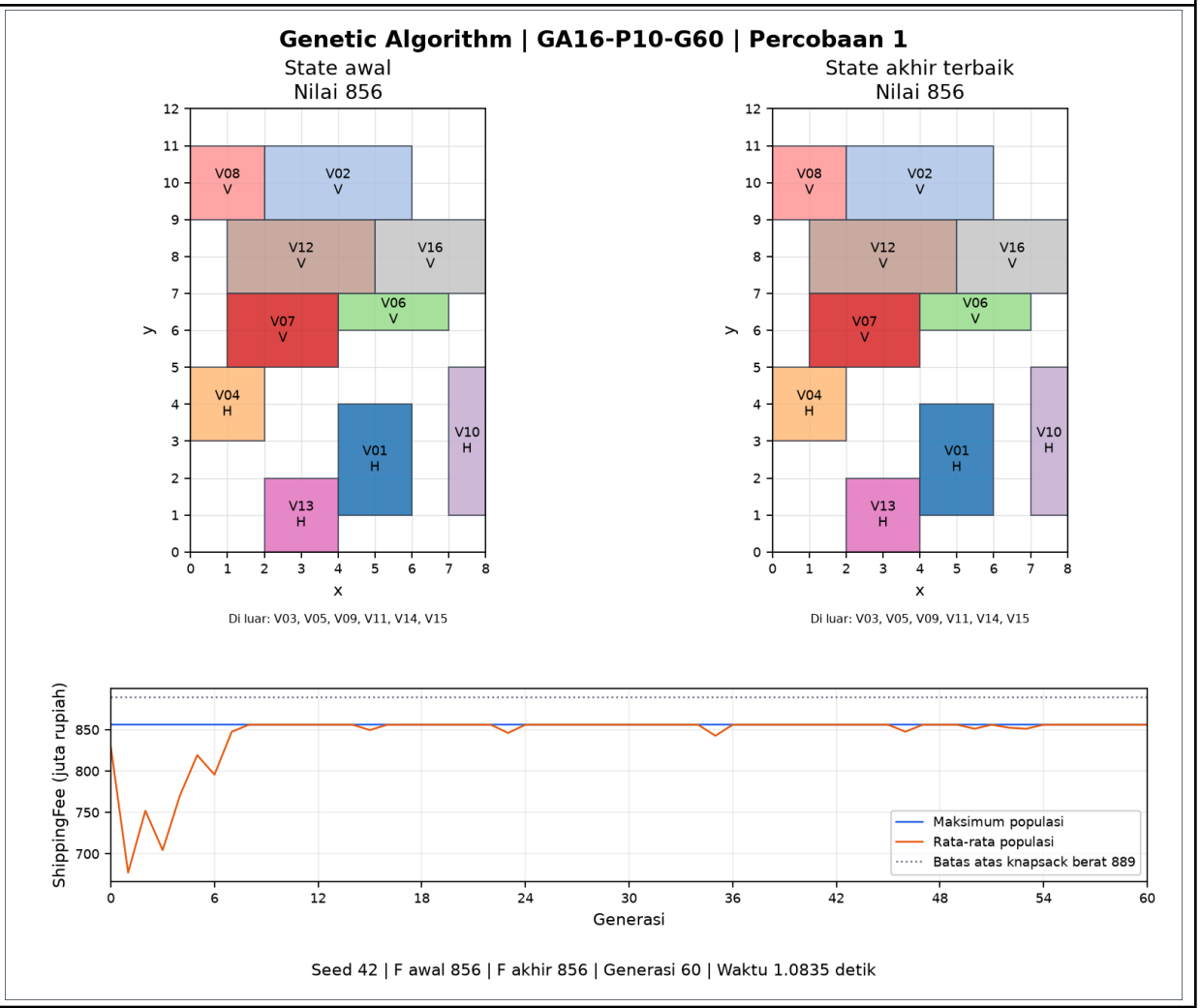
a. Analisis Hasil Percobaan Genetic Algorithm GA_G120

Dari ketiga percobaan, diperoleh rata-rata nilai akhir sebesar 871,00 juta rupiah, dengan nilai terendah 868 dan nilai tertinggi 875. Simpangan baku sampel sebesar 3,61 menunjukkan variasi kualitas akibat pengacakan. Rata-rata durasi pencarian adalah 7,1845 detik.

Elitisme menjaga nilai maksimum populasi agar tidak menurun, sedangkan rata-rata populasi dapat berubah akibat crossover, repair, dan mutasi. Kemiripan nilai antarindividu tidak langsung membuktikan kesamaan susunan kendaraan. Penafsiran konvergensi perlu memperhatikan grafik maksimum serta rata-rata secara bersamaan.

Variasi Jumlah Populasi GA_P10

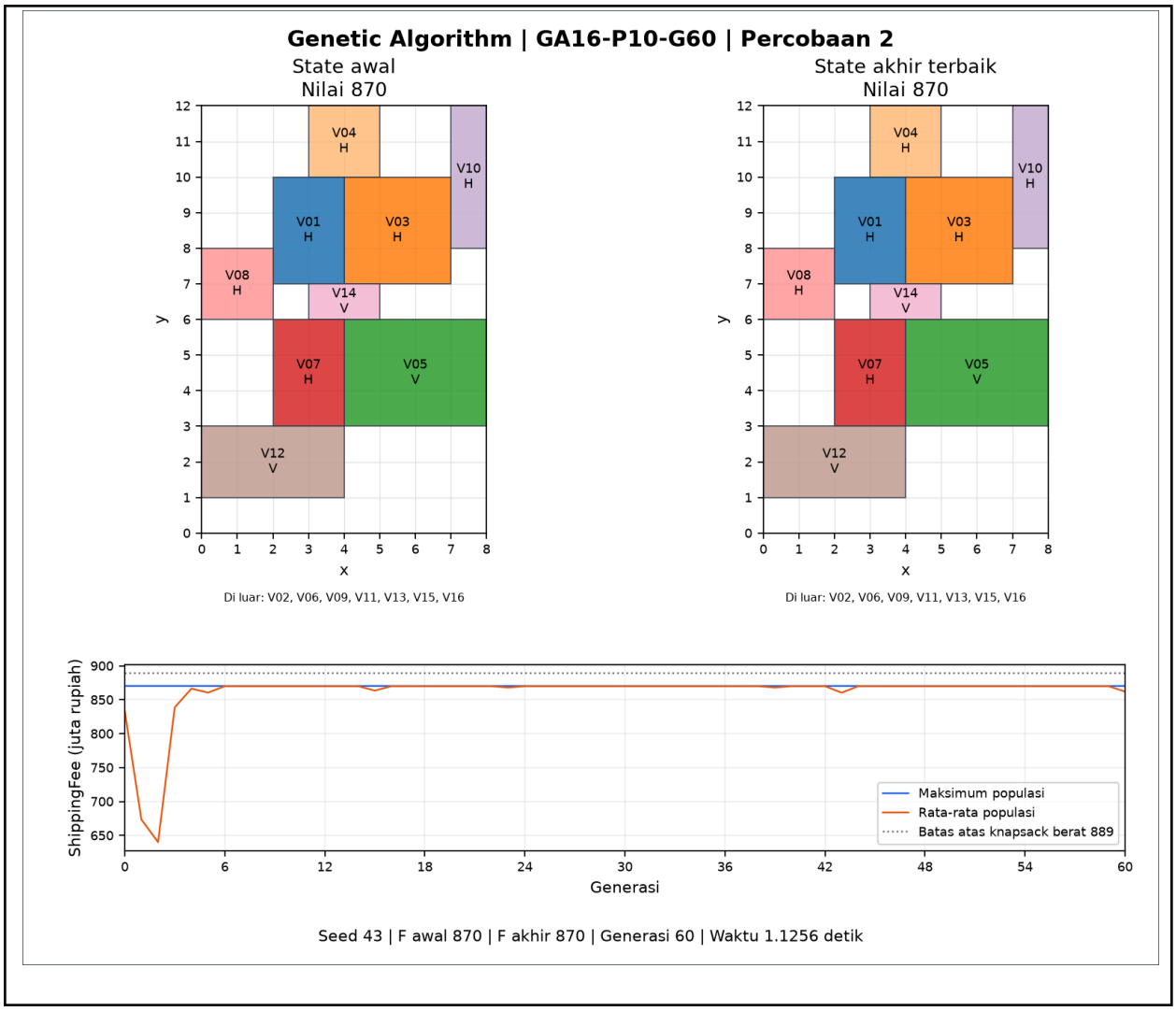
Percobaan Pertama



Gambar 2.19 Genetic Algorithm GA_P10 Percobaan Pertama

Pada percobaan dengan seed 42, nilai *objective function* berubah dari 856 menjadi 856. Pencarian berlangsung selama 1.0835 detik dengan 60 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

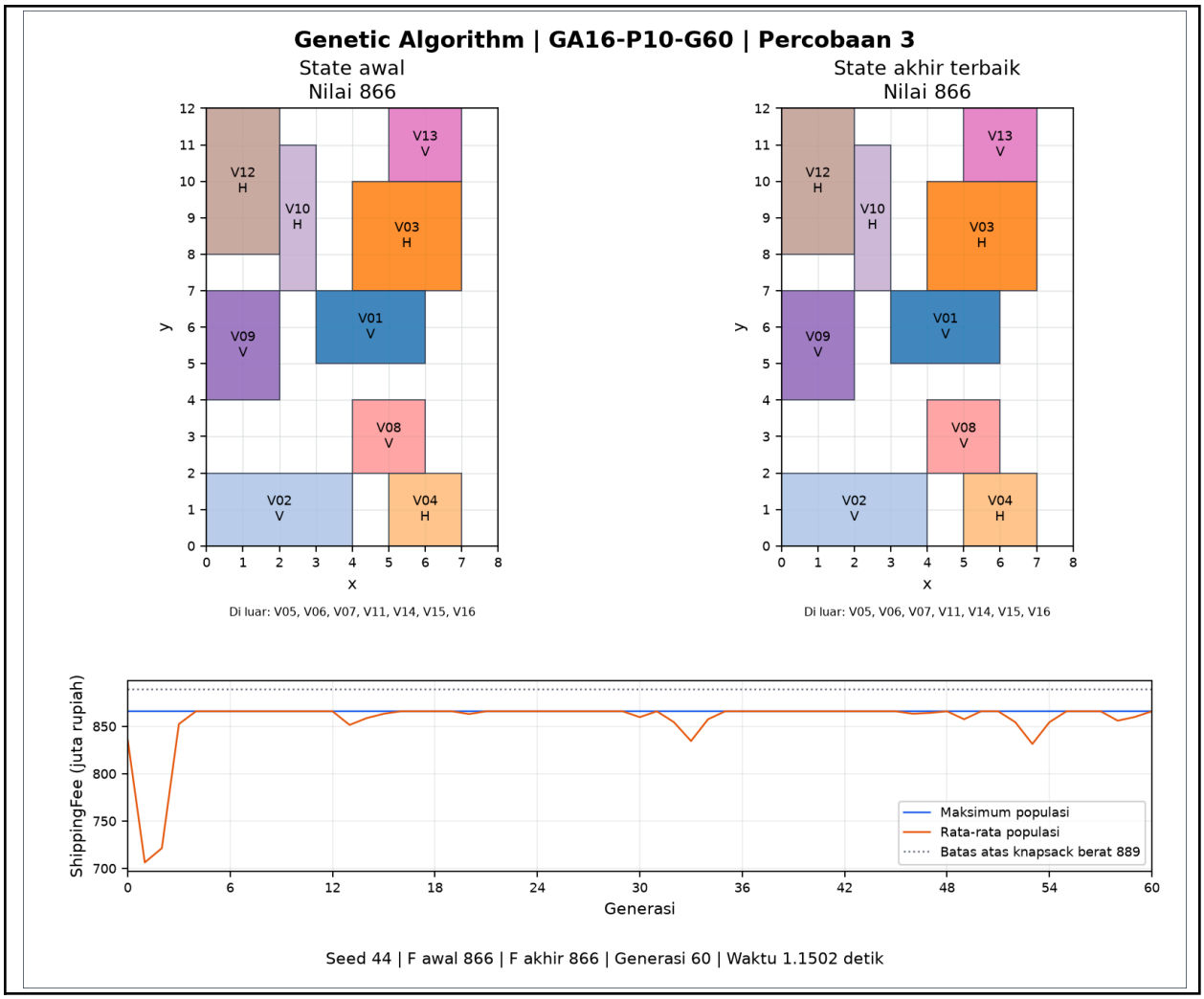
Percobaan Kedua



Gambar 2.20 Genetic Algorithm GA_P10 Percobaan Kedua

Pada percobaan dengan seed 43, nilai *objective function* berubah dari 870 menjadi 870. Pencarian berlangsung selama 1.1256 detik dengan 60 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

Percobaan Ketiga



Gambar 2.21 Genetic Algorithm GA_P10 Percobaan Ketiga

Pada percobaan dengan seed 44, nilai *objective function* berubah dari 866 menjadi 866. Pencarian berlangsung selama 1.1502 detik dengan 60 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

a. Analisis Hasil Percobaan Genetic Algorithm GA_P10

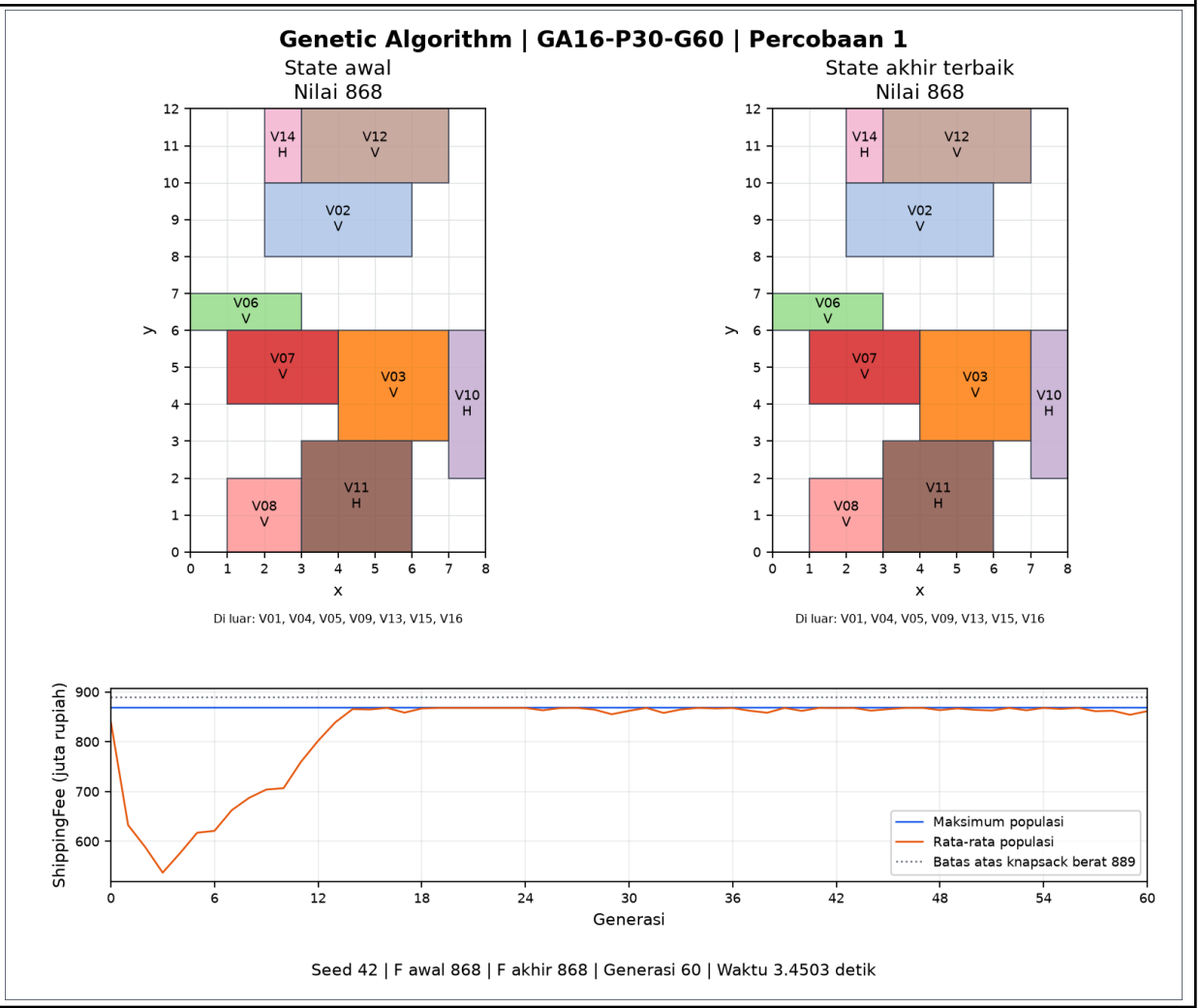
Dari ketiga percobaan, diperoleh rata-rata nilai akhir sebesar 864.00 juta rupiah, dengan nilai terendah 856 dan nilai tertinggi 870. Simpangan baku sampel sebesar 7.21 menunjukkan variasi kualitas akibat pengacakan. Rata-rata durasi pencarian adalah 1.1198 detik.

Elitisme menjaga nilai maksimum populasi agar tidak menurun, sedangkan rata-rata populasi dapat berubah akibat *crossover*, *repair*, dan mutasi. Kemiripan nilai antarindividu tidak

langsung membuktikan kesamaan susunan kendaraan. Penafsiran konvergensi perlu memperhatikan grafik maksimum serta rata-rata secara bersamaan.

Variasi Jumlah Populasi GA_P30

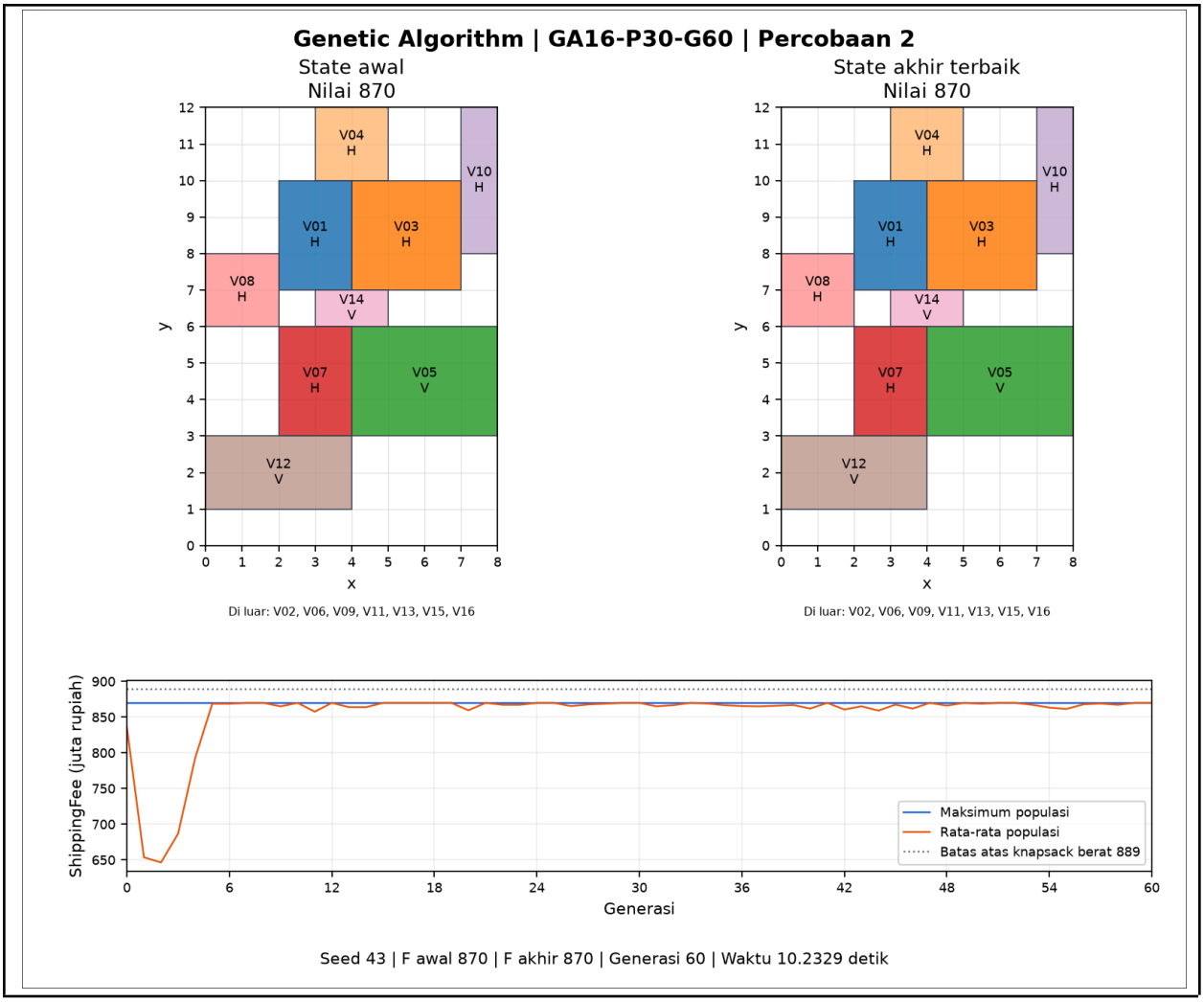
Percobaan Pertama



Gambar 2.22 Genetic Algorithm GA_P30 Percobaan Pertama

Pada percobaan dengan seed 42, nilai *objective function* berubah dari 868 menjadi 868. Pencarian berlangsung selama 3.4503 detik dengan 60 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

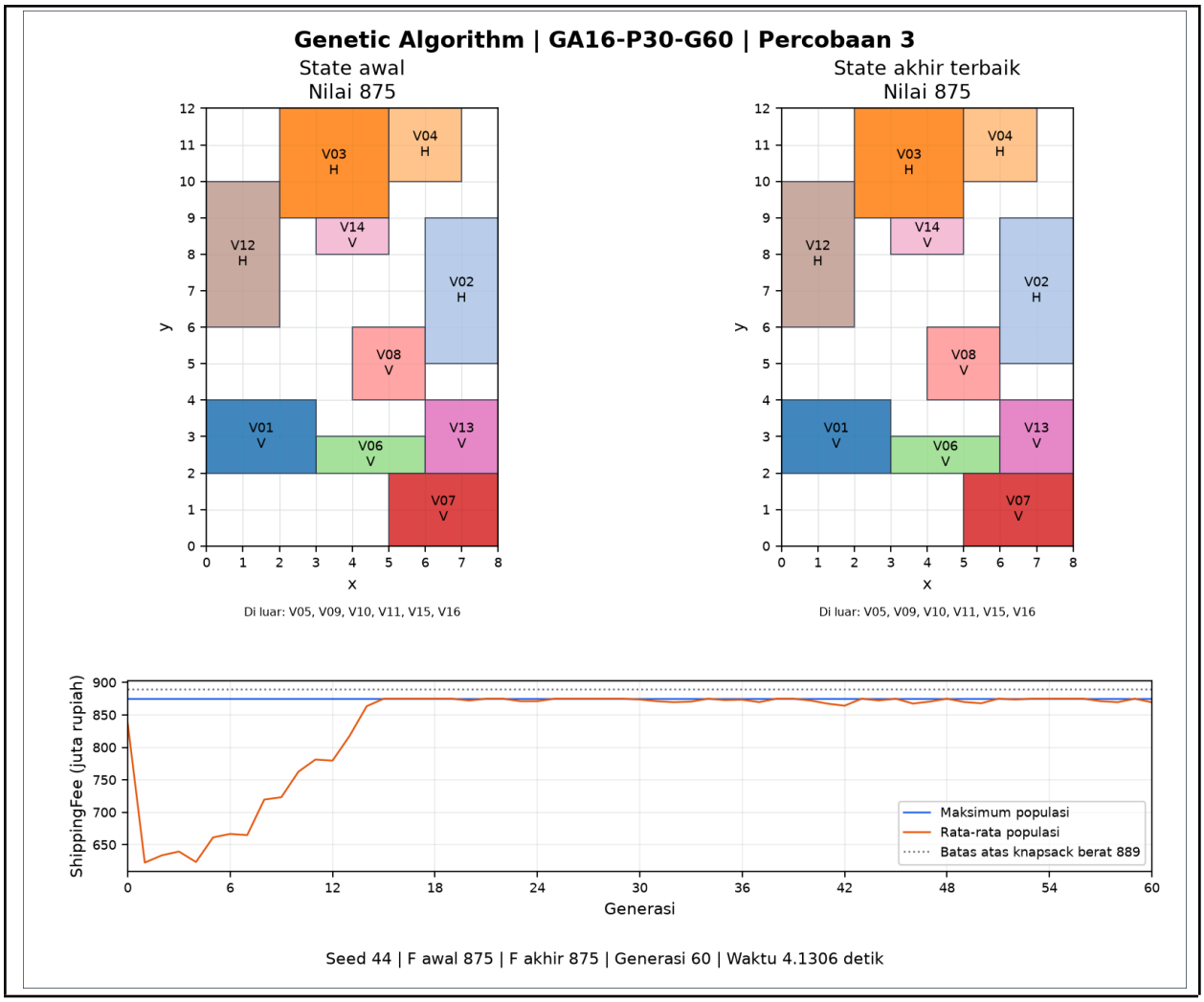
Percobaan Kedua



Gambar 2.23 Genetic Algorithm GA_P30 Percobaan Kedua

Pada percobaan dengan seed 43, nilai *objective function* berubah dari 870 menjadi 870. Pencarian berlangsung selama 10.2329 detik dengan 60 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

Percobaan Ketiga



Gambar 2.24 Genetic Algorithm GA_P30 Percobaan Ketiga

Pada percobaan dengan seed 44, nilai *objective function* berubah dari 875 menjadi 875. Pencarian berlangsung selama 4.1306 detik dengan 60 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

a. Analisis Hasil Percobaan Genetic Algorithm GA_P30

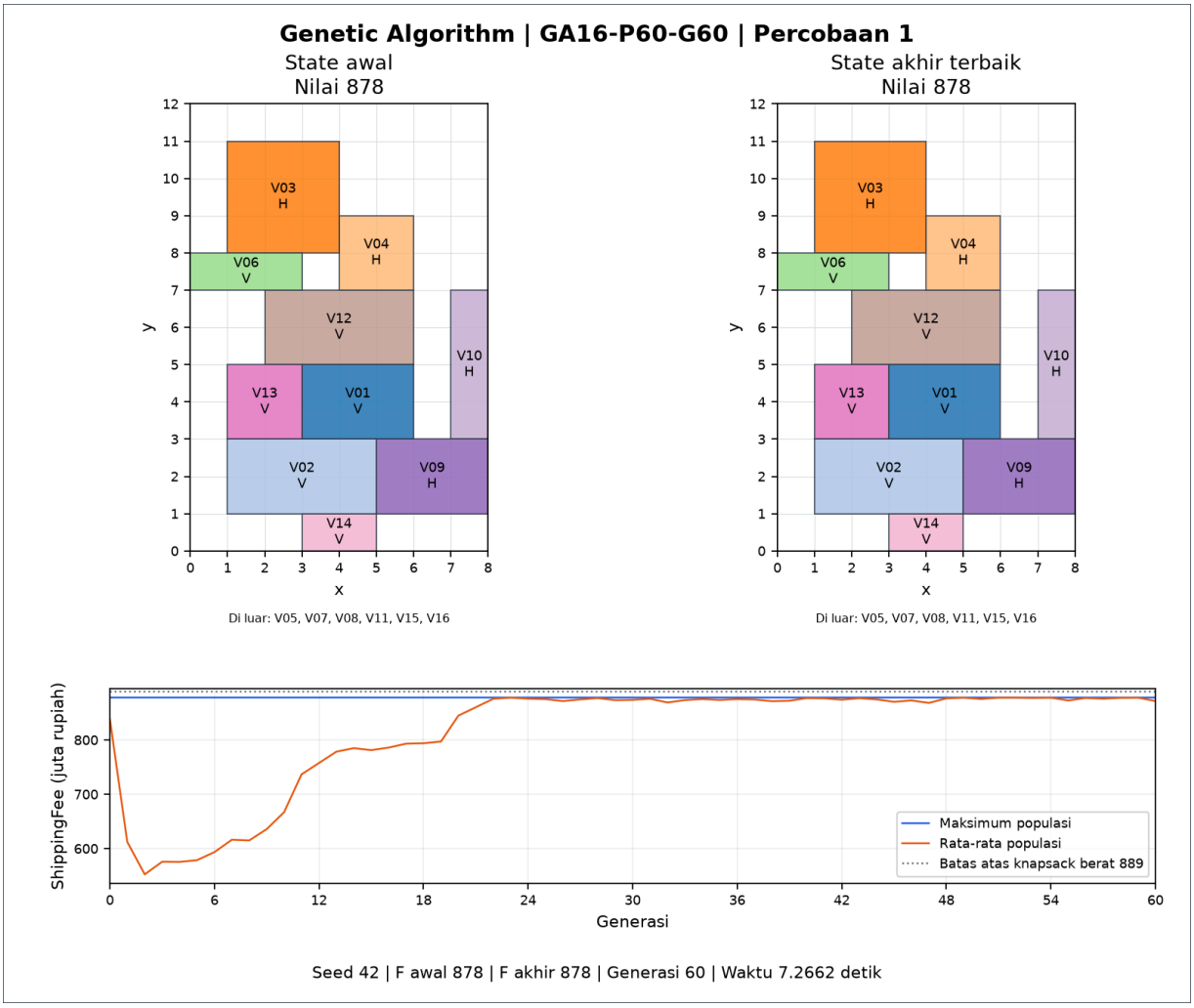
Dari ketiga percobaan, diperoleh rata-rata nilai akhir sebesar 871.00 juta rupiah, dengan nilai terendah 868 dan nilai tertinggi 875. Simpangan baku sampel sebesar 3.61 menunjukkan variasi kualitas akibat pengacakan. Rata-rata durasi pencarian adalah 5,9379 detik.

Elitisme menjaga nilai maksimum populasi agar tidak menurun, sedangkan rata-rata populasi dapat berubah akibat *crossover*, *repair*, dan mutasi. Kemiripan nilai antarindividu tidak

langsung membuktikan kesamaan susunan kendaraan. Penafsiran konvergensi perlu memperhatikan grafik maksimum serta rata-rata secara bersamaan.

Variasi Jumlah Populasi GA_P60

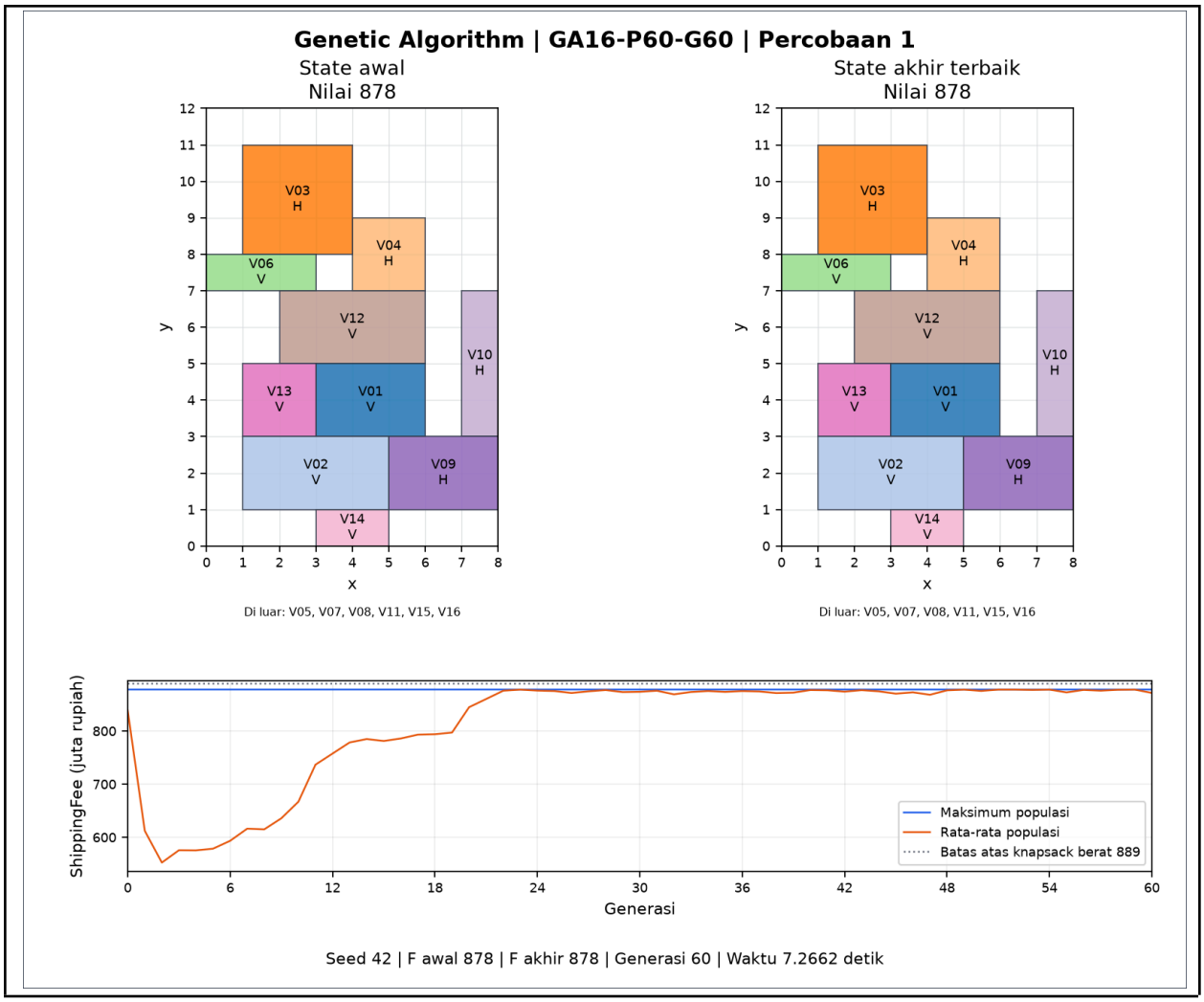
Percobaan Pertama



Gambar 2.25 Genetic Algorithm GA_P60 Percobaan Pertama

Pada percobaan dengan seed 42, nilai *objective function* berubah dari 878 menjadi 878. Pencarian berlangsung selama 7.2662 detik dengan 60 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

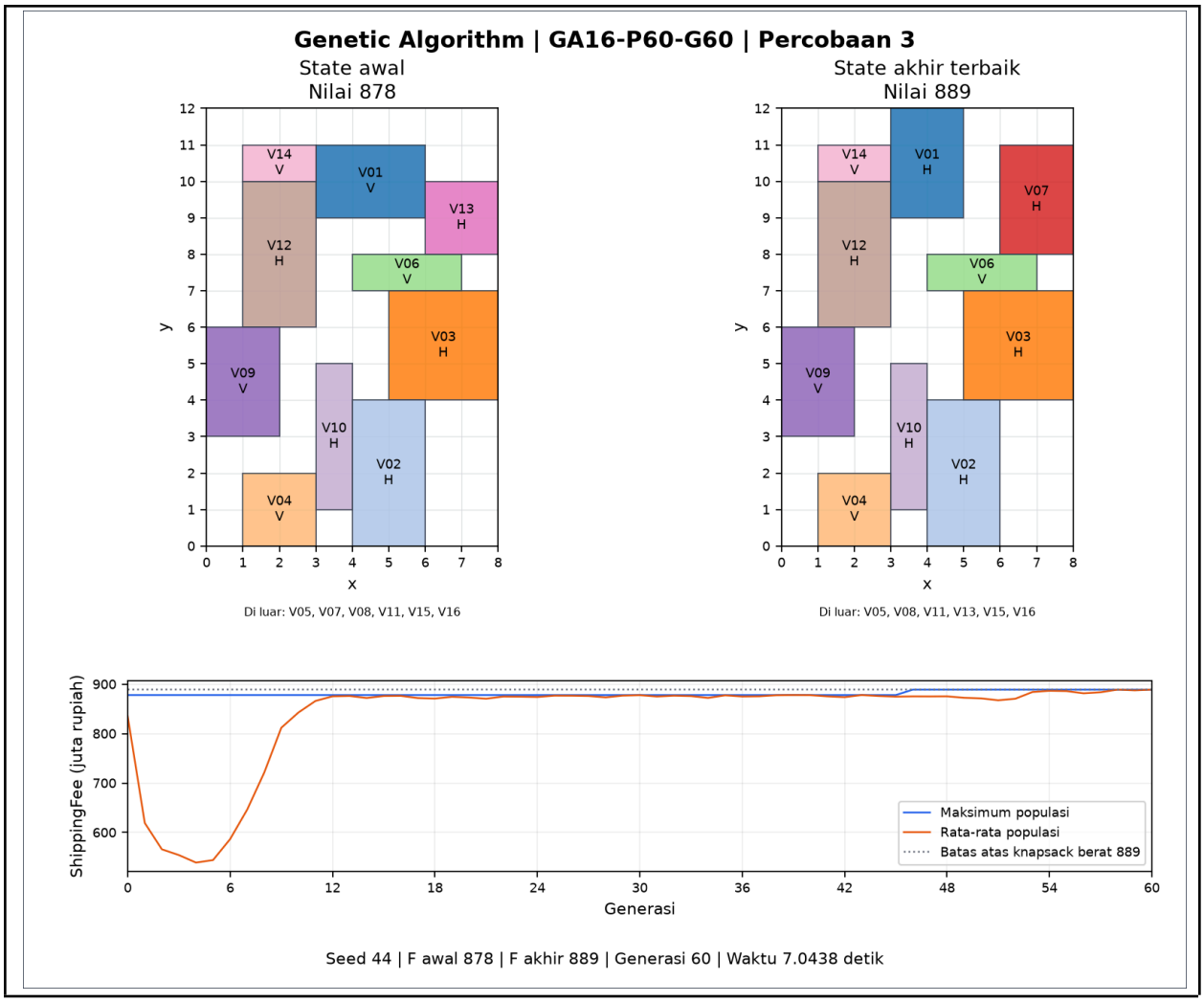
Percobaan Kedua



Gambar 2.26 Genetic Algorithm GA_P60 Percobaan Kedua

Pada percobaan dengan seed 43, nilai *objective function* berubah dari 870 menjadi 877. Pencarian berlangsung selama 7.26662 detik dengan 60 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

Percobaan Ketiga



Gambar 2.27 Genetic Algorithm GA_P60 Percobaan Ketiga

Pada percobaan dengan seed 44, nilai *objective function* berubah dari 878 menjadi 889. Pencarian berlangsung selama 7.0438 detik dengan 60 iterasi. *State* akhir telah memenuhi seluruh *constraint* penempatan kendaraan.

a. Analisis Hasil Percobaan Genetic Algorithm GA_P60

Dari ketiga percobaan, diperoleh rata-rata nilai akhir sebesar 881.33 juta rupiah, dengan nilai terendah 877 dan nilai tertinggi 889. Simpangan baku sampel sebesar 6.66 menunjukkan variasi kualitas akibat pengacakan. Rata-rata durasi pencarian adalah 7.1922 detik.

Elitisme menjaga nilai maksimum populasi agar tidak menurun, sedangkan rata-rata populasi dapat berubah akibat *crossover*, *repair*, dan mutasi. Kemiripan nilai antarindividu tidak

langsung membuktikan kesamaan susunan kendaraan. Penafsiran konvergensi perlu memperhatikan grafik maksimum serta rata-rata secara bersamaan.

b. Analisis Pengaruh Jumlah Populasi dan Banyak Iterasi

GA_G20: rata-rata F 871.00; rata-rata durasi 1.2928 detik.

GA_G60: rata-rata F 871.00; rata-rata durasi 3.6207 detik.

GA_G120: rata-rata F 871.00; rata-rata durasi 7.1845 detik.

Pada seed yang diuji, menambah generasi dari 20 menjadi 60 dan 120 tidak meningkatkan nilai akhir. Ini menunjukkan manfaat tambahan generasi telah habis pada lintasan pencarian yang teramati. Waktu tetap meningkat karena generasi tambahan masih menghasilkan, memperbaiki, dan menilai anak meskipun kandidat terbaik tidak berubah. Hasil ini tidak berarti penambahan generasi selalu tidak bermanfaat untuk instance lain.

GA_P10: rata-rata F 864.00; SD 7.21; rata-rata durasi 1.1198 detik.

GA_P30: rata-rata F 871.00; SD 3.61; rata-rata durasi 5.9379 detik.

GA_P60: rata-rata F 881.33; SD 6.66; rata-rata durasi 7.1922 detik.

Populasi lebih besar memberi lebih banyak kandidat awal dan pasangan parent, tetapi juga meningkatkan biaya generasi. Peningkatan kualitas harus dibaca bersama biaya waktu dan variasi seed. Seed yang sama pada populasi berbeda tidak menghasilkan populasi atau seluruh bilangan acak lanjutan yang sama, sehingga perbedaan run tidak hanya disebabkan ukuran populasi secara deterministik.

2.6 Analisis Perbandingan Algoritma dalam Mendekati Global Optima

Metrik utama adalah F akhir. Metrik pendukung meliputi kenaikan terhadap *state* awal, jumlah kendaraan terangkut, pemakaian berat, okupansi area, iterasi aktual, dan durasi. Untuk konsistensi tiga pengulangan digunakan rata-rata, simpangan baku sampel, minimum, dan maksimum.

Batas atas dihitung melalui knapsack 0/1 dengan kapasitas berat 55 dan semua kendaraan pada data uji. Pembatas geometri dihapus, sedangkan setiap kendaraan tetap boleh dipilih paling banyak satu kali. Karena setiap solusi geometris yang valid juga memenuhi knapsack berat, optimum knapsack menjadi batas atas sah untuk objective penempatan kapal.

Rekurensi yang dipakai adalah DP_i(b) = max(DP_(i−1)(b), DP_(i−1)(b−weight_i) + fee_i) jika berat mencukupi. Implementasi satu array memperbarui kapasitas dari besar ke kecil

agar kendaraan yang sama tidak terpilih berulang. Seluruh berat benchmark adalah bilangan bulat, sehingga kapasitas dapat diindeks langsung.

Hasil dynamic programming adalah U = 889. Untuk solusi F, gap terhadap batas atas didefinisikan (U − F) / U × 100%. Besaran tersebut membatasi kemungkinan kekurangan terhadap optimum, dan saat suatu solusi valid mencapai 889, solusi tersebut terbukti optimal. Pada eksperimen GA_P60 seed 44, kesetaraan ini benar-benar tercapai.

HC berhenti karena aturan sampling dan stagnasi, sehingga tidak dapat dinyatakan mencapai local optimum tersertifikasi. Algoritma heuristik tidak memberikan jaminan umum global optimum. Namun, untuk run GA_P60 seed 44, validitas susunan dan kesamaan nilai terhadap batas atas memberikan bukti optimum untuk benchmark ini. Perbandingan kualitas disimpulkan hanya dari hasil teramati, dengan batas atas digunakan sebagai konteks kuantitatif.

2.7 Analisis Perbandingan Hasil Pencarian Algoritma

Stochastic Hill-Climbing (HC) memperoleh rata-rata pendapatan sebesar 841.33 juta rupiah, dengan rentang nilai akhir 833–856 juta rupiah dan simpangan baku sampel 12.74. Rata-rata waktu pencarian adalah 0.0490 detik. Nilai terbaik mencapai 96.29% dari batas atas knapsack sebesar 889 juta rupiah.

Simulated Annealing (SA) memperoleh rata-rata pendapatan sebesar 868.67 juta rupiah, dengan rentang nilai akhir 863–877 juta rupiah dan simpangan baku sampel 7.37. Rata-rata waktu pencarian adalah 0.3187 detik. Nilai terbaik mencapai 98.65% dari batas atas knapsack.

Sebagai konfigurasi pembanding utama, Genetic Algorithm GA_G60 memperoleh rata-rata pendapatan sebesar 871.00 juta rupiah, dengan rentang nilai akhir 868–875 juta rupiah dan simpangan baku sampel 3.61. Rata-rata waktu pencarian adalah 3.6207 detik. Nilai terbaik mencapai 98.43% dari batas atas knapsack.

HC menerima peningkatan nilai secara ketat sehingga biaya pencariannya relatif kecil, tetapi plateau pendapatan dapat menghambat penataan ulang kendaraan. SA menerima perubahan setara dan sebagian penurunan berdasarkan probabilitas yang dikendalikan temperatur, sehingga dapat menjelajahi konfigurasi yang tidak dapat dilewati HC. GA

mengeksplorasi banyak kandidat secara bersamaan melalui populasi, tetapi proses crossover, repair, dan mutasi turut memengaruhi kualitas solusi serta biaya komputasi.

Pada variasi jumlah generasi, GA_G20, GA_G60, dan GA_G120 menghasilkan rata-rata pendapatan yang sama, yaitu 871.00 juta rupiah. Pada variasi populasi, GA_P10 menghasilkan rata-rata 864.00 juta rupiah, GA_P30 sebesar 871.00 juta rupiah, dan GA_P60 sebesar 881.33 juta rupiah. Hasil terbaik di seluruh percobaan diperoleh oleh GA_P60 pada seed 44 dengan nilai 889 juta rupiah.

Batas atas knapsack untuk benchmark ini adalah 889 juta rupiah. Karena solusi GA_P60 seed 44 memenuhi seluruh constraint dan mencapai nilai yang sama dengan batas atas tersebut, nilai optimum global untuk benchmark ini terbukti sebesar 889 juta rupiah.

Tabel 2.2 Rekapitulasi Hasil Pencarian

Konfigurasi	**F seed 42 / 43 / 44**	**Rata-rata**	**SD**	**Detik**
HC	835 / 856 / 833	841.33	12.74	0.0490
SA	877 / 863 / 866	868.67	7.37	0.3187
GA_G20	868 / 870 / 875	871.00	3.61	1.2928
GA_G60	868 / 870 / 875	871.00	3.61	3.6207
GA_G120	868 / 870 / 875	871.00	3.61	7.1845
GA_P10	856 / 870 / 866	864.00	7.21	1.1198
GA_P30	868 / 870 / 875	871.00	3.61	5.9379
GA_P60	878 / 877 / 889	881.33	6.66	7.1922

2.8 Analisis Perbandingan Durasi Pencarian Algoritma

Durasi pencarian menunjukkan biaya komputasi yang diperlukan setiap konfigurasi untuk menghasilkan solusi pada benchmark yang diuji. HC memiliki rata-rata waktu pencarian 0.0490 detik, sedangkan SA memiliki rata-rata waktu 0.3187 detik. Perbedaan ini menggambarkan biaya eksekusi pada implementasi dan lingkungan pengujian yang digunakan, bukan perbandingan dengan jumlah evaluasi objective yang disamakan.

Pada variasi jumlah generasi, GA_G20 memiliki rata-rata durasi 1.2928 detik, GA_G60 sebesar 3.6207 detik, dan GA_G120 sebesar 7.1845 detik. Ketiga konfigurasi menghasilkan rata-rata objective yang sama, yaitu 871.00 juta rupiah. Pada tiga seed yang diuji, penambahan generasi tidak meningkatkan nilai akhir rata-rata, tetapi meningkatkan waktu komputasi.

Pada variasi ukuran populasi, GA_P10 memiliki rata-rata durasi 1.1198 detik dan rata-rata objective 864.00 juta rupiah. GA_P30 memiliki rata-rata durasi 5.9379 detik dengan rata-rata objective 871.00 juta rupiah. GA_P60 memiliki rata-rata durasi 7.1922 detik dengan rata-rata objective 881.33 juta rupiah. Waktu GA_P30 dipengaruhi oleh percobaan kedua yang tercatat selama 10.2329 detik, sehingga rata-ratanya relatif tinggi dibandingkan percobaan pertama dan ketiga.

GA_G60 dan GA_P30 menggunakan kombinasi parameter yang sama, yaitu 60 generasi dan populasi 30, tetapi berasal dari dua rangkaian eksperimen yang terpisah. Keduanya tetap ditampilkan sebagai dua rangkaian kontrol dan tidak digabung sebagai enam pengulangan independen. Nilai objective akhir yang tercatat sama untuk setiap seed, sementara durasi dapat berbeda di antara eksekusi. Karena tidak dilakukan benchmark waktu khusus dengan pengulangan terkontrol, perbedaan tersebut tidak dapat dipastikan hanya disebabkan oleh spesifikasi laptop atau satu faktor tertentu.

Durasi GA mencakup pembentukan populasi awal di dalam fungsi algoritma, sedangkan pembentukan state awal HC dan SA dilakukan sebelum timer. Oleh karena itu, perbandingan durasi lintas algoritma menggambarkan biaya eksekusi sesuai prosedur pengukuran yang dipakai, bukan efisiensi per evaluasi objective yang setara.

Simpangan baku sampel dihitung menggunakan penyebut $n-1$ untuk tiga pengulangan. Interpretasi kinerja tetap dibatasi pada benchmark, seed, dan konfigurasi yang diuji.

2.9 Analisis Perbandingan Konsistensi Algoritma

Seluruh 24 state akhir yang dilaporkan memenuhi constraint penempatan kendaraan. Pada dua rangkaian eksperimen GA dengan parameter kontrol yang sama, nilai objective akhir

dilaporkan sama untuk setiap seed. Konsistensi nilai objective dan history perlu dibedakan dari konsistensi durasi: durasi eksekusi dapat bervariasi, sementara nilai objective akhir tetap sama.

Hasil eksperimen memperlihatkan bahwa konsistensi kelayakan tidak selalu berarti konsistensi kualitas. Seluruh keluaran dapat memenuhi constraint, tetapi pendapatan akhir tetap berubah antar-seed. Pada GA_P60, misalnya, nilai akhir yang diperoleh adalah 878, 877, dan 889 juta rupiah. Variasi ini menunjukkan bahwa hasil algoritma berbasis pengacakan perlu diringkas dengan rata-rata, rentang, dan simpangan baku sampel.

Eksperimen menggunakan satu benchmark dan tiga seed pada setiap konfigurasi. Dataset terkontrol tersebut belum mewakili seluruh distribusi kendaraan perusahaan. Karena itu, hasil tidak dapat digunakan untuk menyimpulkan dominasi satu algoritma secara universal. Generalisasi memerlukan tambahan instance yang mewakili kendala area dominan, berat dominan, dan ukuran kendaraan yang seragam.

Anggaran iterasi tidak sama dengan anggaran komputasi. Satu generasi GA membentuk banyak individu, sedangkan satu iterasi HC atau SA mengambil satu neighbor valid. Selain itu, random_neighbor dapat membutuhkan beberapa upaya sebelum memperoleh kandidat valid. Jumlah upaya tersebut tidak dicatat, sehingga durasi tidak dapat diuraikan berdasarkan jenis operasi.

Metrik stagnasi SA mengukur episode best-so-far yang tidak mencatat rekor baru, bukan local optimum yang dibuktikan secara matematis. Eksperimen ini juga tidak mencakup enumerasi seluruh neighbor, solver geometri eksak, pengujian signifikansi statistik, atau pengukuran penggunaan memori. Perbaikan berikutnya dapat mengevaluasi repair berbasis prioritas tarif atau densitas nilai, variasi mutasi, pencatatan jumlah evaluasi, dan mekanisme penataan ulang plateau dengan protokol perbandingan yang konsisten.

BAB III
KESIMPULAN DAN SARAN

3.1 Kesimpulan

Representasi *State* dengan koordinat opsional dapat memodelkan kendaraan di dalam dan di luar geladak secara eksplisit. Pemisahan validator, evaluator, serta operator *neighbor* membantu menjaga pencarian pada *state* legal. Fungsi objektif memaksimalkan total *ShippingFee*; okupansi dan ETA tidak secara langsung menentukan nilai pencarian.

Eksperimen menghasilkan solusi terbaik 889 juta rupiah dan membatasi optimum pada interval [889, 889]. Seluruh *state* akhir memenuhi batas kapal, bebas overlap, dan tidak melampaui kapasitas. Gap solusi terbaik terhadap batas atas adalah 0.00%.

HC efisien untuk perbaikan cepat, tetapi penerimaan ketat membatasi perlintasan *plateau*. SA memberikan jalur eksplorasi melalui penerimaan penurunan yang dikendalikan temperatur. GA memberi pencarian berbasis populasi dan elitisme, tetapi biaya komputasinya dipengaruhi jumlah populasi dan generasi, serta perilaku *repair*.

Penambahan generasi GA pada benchmark ini tidak memperbaiki nilai akhir setelah 20 generasi pada seed yang diuji. Pemilihan parameter karena itu perlu mempertimbangkan titik berhenti yang efisien, bukan hanya menggunakan nilai parameter terbesar. Kesimpulan ini bersifat empiris untuk benchmark dan konfigurasi yang diuji.

3.2 Saran

Perluasan eksperimen sebaiknya mencakup lebih banyak instance dan seed, pencatatan evaluasi objective, serta perbandingan dengan anggaran waktu yang setara. Untuk instance yang belum mencapai batas atas, solver eksak dapat dipakai untuk mengetahui optimum geometris. Untuk penerapan operasional, model juga perlu mempertimbangkan akses bongkar, urutan kedatangan, dan distribusi muatan bila kebutuhan tersebut ditetapkan.

BAB IV
PEMBAGIAN TUGAS TIAP ANGGOTA KELOMPOK

Pembagian tugas berikut merupakan rancangan pembagian kerja yang perlu disesuaikan dengan kontribusi aktual setiap anggota kelompok.

Tabel 4.1 Pembagian Tugas Kelompok

NIM	**Nama**	**Pembagian Tugas**
18224120	Ibrahim Ferizarizqi Permana	Integrasi model, objective dan constraints; peninjauan hasil eksperimen dan laporan
18224030	Adham Sachadeva Purwadi	Peninjauan Stochastic Hill-Climbing dan Simulated Annealing; validasi parameter dan laporan
18224112	Muhammad Reyna Athallah Agoes	Peninjauan Genetic Algorithm, visualisasi serta analisis laporan

REFERENSI

[1] Tim Asisten Laboratorium Inteligensi Artifisial. Tugas Besar 1 IF3070 Dasar Inteligensi Artifisial 2026/2027. Versi 1.0, 24 September 2026. Dokumen lokal SPEK TUBES MUNIR.pdf. Rujukan untuk deskripsi persoalan, batasan, dan rancangan eksperimen.

[2] Proyek IF3070_G15_munir. Modul core/models.py, placement.py, *constraint*s.py, objective.py, initial_*state*.py, dan *neighbor*s.py. Sumber implementasi yang diperiksa pada 21 Oktober 2026.

[3] Proyek IF3070_G15_munir. Modul algorithms/hill_climbing.py dan algorithms/genetic_algorithm.py. Sumber implementasi algoritma dan konfigurasi hasil pada laporan.

[4] Kelompok 15 Munir. experiments.py dan build_report.py. Implementasi pendamping *Simulated Annealing*, benchmark, eksperimen, dan penyajian hasil. Disiapkan dengan bantuan generative AI pada 21 Oktober 2026.

[5] Laporan contoh Tubes santAI mengenai Bin Packing Problem, 66 halaman. Digunakan sebagai acuan struktur penyajian dan tingkat perincian, tanpa mengadopsi data eksperimen maupun pembagian kerja dari kasus contoh.

[6] Russell, S., dan Norvig, P. Artificial Intelligence: A Modern Approach. Edisi ke-4. Pearson, 2021. Rujukan untuk local search, hill-climbing, serta pembahasan plateau dan local optimum. Tersedia: https://aima.cs.berkeley.edu/.

[7] Kirkpatrick, S., Gelatt, C. D., Jr., dan Vecchi, M. P. Optimization by Simulated Annealing. Science, vol. 220, no. 4598, hlm. 671–680, 1983. Rujukan untuk temperatur, pendinginan, dan penerimaan kandidat dengan nilai lebih rendah pada Simulated Annealing. DOI: https://doi.org/10.1126/science.220.4598.671.

[8] Mitchell, M. An Introduction to Genetic Algorithms. MIT Press, 1996. Rujukan untuk representasi individu, populasi, seleksi, crossover, dan mutasi pada Genetic Algorithm. DOI: https://doi.org/10.7551/mitpress/3927.001.0001.

[9] Kellerer, H., Pferschy, U., dan Pisinger, D. Knapsack Problems. Springer, 2004. Rujukan untuk masalah knapsack 0/1 dan dynamic programming yang digunakan dalam perhitungan batas atas pendapatan. DOI: https://doi.org/10.1007/978-3-540-24777-7.

Referensi [1] menentukan kebutuhan masalah, sedangkan referensi [2] dan [3] menentukan perilaku kode yang benar-benar dieksekusi. Perbedaan antara spesifikasi dan

implementasi dicatat dalam uraian: khususnya ETA yang belum memengaruhi objective serta penambahan SA sebagai pendamping. Hash SHA-256 modul tersimpan pada manifest.json untuk menandai versi sumber eksperimen.

LAMPIRAN A DATA UJI DAN PENGGUNAAN AI

Tabel A.1 Rincian Data Uji Kendaraan

ID	Dimensi	Tarif	Berat	ETA
V01	2 × 3	80	5	1
V02	2 × 4	115	7	2
V03	3 × 3	145	9	3
V04	2 × 2	65	4	4
V05	3 × 4	180	12	5
V06	1 × 3	48	3	1
V07	2 × 3	95	6	2
V08	2 × 2	72	5	3
V09	3 × 2	108	7	4
V10	1 × 4	62	4	5
V11	3 × 3	160	11	1
V12	2 × 4	135	8	2
V13	2 × 2	84	6	3
V14	1 × 2	36	2	4
V15	3 × 4	195	13	5
V16	2 × 3	100	7	1

Lampiran ini mencatat penggunaan AI dalam penyusunan laporan dan eksperimen. Isinya merupakan catatan faktual proses bantuan, bukan pengganti lembar resmi atau tanda tangan anggota. Formulir resmi institusi perlu diisi dan disahkan oleh kelompok sesuai format yang berlaku.

Data eksperimen dihasilkan melalui eksekusi fungsi pencarian. Benchmark kendaraan dibuat untuk pengujian terkontrol dan tidak dinyatakan sebagai data lapangan. Kontribusi individu tidak disimpulkan dari keluaran AI; tabel pembagian tugas sebelumnya tetap berstatus usulan sampai diverifikasi anggota.

Kelompok perlu membaca ulang source code dan hasil, memastikan setiap anggota memahami algoritma, memperbarui pembagian kerja aktual, serta melengkapi formulir penggunaan AI yang diwajibkan. Tidak ada persetujuan atau tanda tangan anggota yang dibuat secara otomatis pada laporan ini.

LAMPIRAN B RINCIAN STATE PERCOBAAN

Tabel berikut menunjukkan koordinat dan orientasi setiap kendaraan pada *initial state* dan *final state*. Simbol H berarti horizontal dan V berarti vertikal. Kendaraan yang tidak diangkut ditulis sebagai Di luar. Pada GA, *state* awal merupakan individu terbaik generasi nol.

Tabel B.1 Penempatan Kendaraan HC Seed 42

Kendaraan	**State Awal**	**State Akhir**
V01	Di luar	(6, 4) H
V02	Di luar	Di luar
V03	Di luar	Di luar
V04	Di luar	(1, 6) V
V05	Di luar	Di luar
V06	(2, 2) H	(2, 2) H
V07	(3, 6) V	(3, 6) V
V08	(6, 4) H	Di luar
V09	(6, 0) V	(6, 0) V
V10	(1, 6) H	Di luar
V11	(3, 0) H	(3, 0) H
V12	Di luar	Di luar
V13	(3, 4) H	(3, 4) H
V14	Di luar	Di luar
V15	(4, 8) H	(4, 8) H
V16	Di luar	Di luar

Nilai awal 824 juta rupiah; nilai akhir 835 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.2 Penempatan Kendaraan HC Seed 43

Kendaraan	**State Awal**	**State Akhir**
V01	(1, 6) V	(1, 6) V
V02	Di luar	(3, 2) V
V03	Di luar	Di luar
V04	(0, 9) H	(0, 9) H
V05	Di luar	Di luar
V06	(0, 6) H	(0, 6) H
V07	(4, 9) H	(4, 9) H
V08	Di luar	Di luar
V09	(3, 2) H	Di luar
V10	(3, 4) V	(3, 4) V
V11	(5, 6) H	(5, 6) H
V12	Di luar	Di luar
V13	Di luar	Di luar
V14	(7, 4) H	(7, 4) H
V15	(0, 1) H	(0, 1) H
V16	Di luar	Di luar

Nilai awal 849 juta rupiah; nilai akhir 856 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.3 Penempatan Kendaraan HC Seed 44

Kendaraan	**State Awal**	**State Akhir**
V01	(2, 8) H	(2, 8) H
V02	Di luar	Di luar
V03	Di luar	Di luar
V04	Di luar	Di luar
V05	(4, 5) V	(4, 5) V
V06	(3, 2) V	(3, 2) V
V07	Di luar	(4, 8) H
V08	(3, 0) H	(3, 0) H
V09	Di luar	Di luar
V10	(1, 6) H	(1, 6) H
V11	(1, 3) H	(1, 3) H
V12	Di luar	Di luar
V13	(4, 8) V	Di luar
V14	(7, 10) H	(7, 10) H
V15	Di luar	Di luar
V16	(6, 1) H	(6, 1) H

Nilai awal 822 juta rupiah; nilai akhir 833 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.4 Penempatan Kendaraan SA Seed 42

Kendaraan	State Awal	State Akhir
V01	Di luar	(3, 0) V
V02	Di luar	(4, 5) V
V03	Di luar	(1, 2) V
V04	Di luar	(4, 8) V
V05	Di luar	Di luar
V06	(2, 2) H	Di luar
V07	(3, 6) V	(5, 3) V
V08	(6, 4) H	(6, 1) H
V09	(6, 0) V	(2, 5) V
V10	(1, 6) H	(0, 6) H
V11	(3, 0) H	Di luar
V12	Di luar	(6, 8) H
V13	(3, 4) H	Di luar
V14	Di luar	Di luar
V15	(4, 8) H	Di luar
V16	Di luar	Di luar

Nilai awal 824 juta rupiah; nilai akhir 877 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.5 Penempatan Kendaraan SA Seed 43

Kendaraan	State Awal	State Akhir
V01	(1, 6) V	Di luar
V02	Di luar	(5, 6) H
V03	Di luar	(5, 3) H
V04	(0, 9) H	(1, 0) H
V05	Di luar	Di luar
V06	(0, 6) H	(2, 10) V
V07	(4, 9) H	Di luar
V08	Di luar	Di luar
V09	(3, 2) H	Di luar
V10	(3, 4) V	Di luar
V11	(5, 6) H	(0, 6) V
V12	Di luar	(0, 2) H
V13	Di luar	Di luar
V14	(7, 4) H	Di luar
V15	(0, 1) H	(3, 0) V
V16	Di luar	Di luar

Nilai awal 849 juta rupiah; nilai akhir 863 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.6 Penempatan Kendaraan SA Seed 44

Kendaraan	**State Awal**	**State Akhir**
V01	(2, 8) H	(4, 5) V
V02	Di luar	(4, 0) V
V03	Di luar	(0, 6) H
V04	Di luar	Di luar
V05	(4, 5) V	Di luar
V06	(3, 2) V	Di luar
V07	Di luar	Di luar
V08	(3, 0) H	Di luar
V09	Di luar	Di luar
V10	(1, 6) H	Di luar
V11	(1, 3) H	(1, 3) H
V12	Di luar	(6, 7) H
V13	(4, 8) V	Di luar
V14	(7, 10) H	(4, 9) V
V15	Di luar	(0, 0) V
V16	(6, 1) H	Di luar

Nilai awal 822 juta rupiah; nilai akhir 866 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.7 Penempatan Kendaraan GA_G20 Seed 42

Kendaraan	**State Awal**	**State Akhir**
V01	Di luar	Di luar
V02	(2, 8) V	(2, 8) V
V03	(4, 3) V	(4, 3) V
V04	Di luar	Di luar
V05	Di luar	Di luar
V06	(0, 6) V	(0, 6) V
V07	(1, 4) V	(1, 4) V
V08	(1, 0) V	(1, 0) V
V09	Di luar	Di luar
V10	(7, 2) H	(7, 2) H
V11	(3, 0) H	(3, 0) H
V12	(3, 10) V	(3, 10) V
V13	Di luar	Di luar
V14	(2, 10) H	(2, 10) H
V15	Di luar	Di luar
V16	Di luar	Di luar

Nilai awal 868 juta rupiah; nilai akhir 868 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.8 Penempatan Kendaraan GA_G20 Seed 43

Kendaraan	**State Awal**	**State Akhir**
V01	(2, 7) H	(2, 7) H
V02	Di luar	Di luar
V03	(4, 7) H	(4, 7) H
V04	(3, 10) H	(3, 10) H
V05	(4, 3) V	(4, 3) V
V06	Di luar	Di luar
V07	(2, 3) H	(2, 3) H
V08	(0, 6) H	(0, 6) H
V09	Di luar	Di luar
V10	(7, 8) H	(7, 8) H
V11	Di luar	Di luar
V12	(0, 1) V	(0, 1) V
V13	Di luar	Di luar
V14	(3, 6) V	(3, 6) V
V15	Di luar	Di luar
V16	Di luar	Di luar

Nilai awal 870 juta rupiah; nilai akhir 870 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.9 Penempatan Kendaraan GA_G20 Seed 44

Kendaraan	**State Awal**	**State Akhir**
V01	(0, 2) V	(0, 2) V
V02	(6, 5) H	(6, 5) H
V03	(2, 9) H	(2, 9) H
V04	(5, 10) H	(5, 10) H
V05	Di luar	Di luar
V06	(3, 2) V	(3, 2) V
V07	(5, 0) V	(5, 0) V
V08	(4, 4) V	(4, 4) V
V09	Di luar	Di luar
V10	Di luar	Di luar
V11	Di luar	Di luar
V12	(0, 6) H	(0, 6) H
V13	(6, 2) V	(6, 2) V
V14	(3, 8) V	(3, 8) V
V15	Di luar	Di luar
V16	Di luar	Di luar

Nilai awal 875 juta rupiah; nilai akhir 875 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.10 Penempatan Kendaraan GA_G60 Seed 42

Kendaraan	State Awal	State Akhir
V01	Di luar	Di luar
V02	(2, 8) V	(2, 8) V
V03	(4, 3) V	(4, 3) V
V04	Di luar	Di luar
V05	Di luar	Di luar
V06	(0, 6) V	(0, 6) V
V07	(1, 4) V	(1, 4) V
V08	(1, 0) V	(1, 0) V
V09	Di luar	Di luar
V10	(7, 2) H	(7, 2) H
V11	(3, 0) H	(3, 0) H
V12	(3, 10) V	(3, 10) V
V13	Di luar	Di luar
V14	(2, 10) H	(2, 10) H
V15	Di luar	Di luar
V16	Di luar	Di luar

Nilai awal 868 juta rupiah; nilai akhir 868 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.11 Penempatan Kendaraan GA_G60 Seed 43

Kendaraan	**State Awal**	**State Akhir**
V01	(2, 7) H	(2, 7) H
V02	Di luar	Di luar
V03	(4, 7) H	(4, 7) H
V04	(3, 10) H	(3, 10) H
V05	(4, 3) V	(4, 3) V
V06	Di luar	Di luar
V07	(2, 3) H	(2, 3) H
V08	(0, 6) H	(0, 6) H
V09	Di luar	Di luar
V10	(7, 8) H	(7, 8) H
V11	Di luar	Di luar
V12	(0, 1) V	(0, 1) V
V13	Di luar	Di luar
V14	(3, 6) V	(3, 6) V
V15	Di luar	Di luar
V16	Di luar	Di luar

Nilai awal 870 juta rupiah; nilai akhir 870 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.12 Penempatan Kendaraan GA_G60 Seed 44

Kendaraan	**State Awal**	**State Akhir**
V01	(0, 2) V	(0, 2) V
V02	(6, 5) H	(6, 5) H
V03	(2, 9) H	(2, 9) H
V04	(5, 10) H	(5, 10) H
V05	Di luar	Di luar
V06	(3, 2) V	(3, 2) V
V07	(5, 0) V	(5, 0) V
V08	(4, 4) V	(4, 4) V
V09	Di luar	Di luar
V10	Di luar	Di luar
V11	Di luar	Di luar
V12	(0, 6) H	(0, 6) H
V13	(6, 2) V	(6, 2) V
V14	(3, 8) V	(3, 8) V
V15	Di luar	Di luar
V16	Di luar	Di luar

Nilai awal 875 juta rupiah; nilai akhir 875 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.13 Penempatan Kendaraan GA_G120 Seed 42

Kendaraan	**State Awal**	**State Akhir**
V01	Di luar	Di luar
V02	(2, 8) V	(2, 8) V
V03	(4, 3) V	(4, 3) V
V04	Di luar	Di luar
V05	Di luar	Di luar
V06	(0, 6) V	(0, 6) V
V07	(1, 4) V	(1, 4) V
V08	(1, 0) V	(1, 0) V
V09	Di luar	Di luar
V10	(7, 2) H	(7, 2) H
V11	(3, 0) H	(3, 0) H
V12	(3, 10) V	(3, 10) V
V13	Di luar	Di luar
V14	(2, 10) H	(2, 10) H
V15	Di luar	Di luar
V16	Di luar	Di luar

Nilai awal 868 juta rupiah; nilai akhir 868 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.14 Penempatan Kendaraan GA_G120 Seed 43

Kendaraan	**State Awal**	**State Akhir**
V01	(2, 7) H	(2, 7) H
V02	Di luar	Di luar
V03	(4, 7) H	(4, 7) H
V04	(3, 10) H	(3, 10) H
V05	(4, 3) V	(4, 3) V
V06	Di luar	Di luar
V07	(2, 3) H	(2, 3) H
V08	(0, 6) H	(0, 6) H
V09	Di luar	Di luar
V10	(7, 8) H	(7, 8) H
V11	Di luar	Di luar
V12	(0, 1) V	(0, 1) V
V13	Di luar	Di luar
V14	(3, 6) V	(3, 6) V
V15	Di luar	Di luar
V16	Di luar	Di luar

Nilai awal 870 juta rupiah; nilai akhir 870 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.15 Penempatan Kendaraan GA_G120 Seed 44

Kendaraan	**State Awal**	**State Akhir**
V01	(0, 2) V	(0, 2) V
V02	(6, 5) H	(6, 5) H
V03	(2, 9) H	(2, 9) H
V04	(5, 10) H	(5, 10) H
V05	Di luar	Di luar
V06	(3, 2) V	(3, 2) V
V07	(5, 0) V	(5, 0) V
V08	(4, 4) V	(4, 4) V
V09	Di luar	Di luar
V10	Di luar	Di luar
V11	Di luar	Di luar
V12	(0, 6) H	(0, 6) H
V13	(6, 2) V	(6, 2) V
V14	(3, 8) V	(3, 8) V
V15	Di luar	Di luar
V16	Di luar	Di luar

Nilai awal 875 juta rupiah; nilai akhir 875 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.16 Penempatan Kendaraan GA_P10 Seed 42

Kendaraan	State Awal	State Akhir
V01	(4, 1) H	(4, 1) H
V02	(2, 9) V	(2, 9) V
V03	Di luar	Di luar
V04	(0, 3) H	(0, 3) H
V05	Di luar	Di luar
V06	(4, 6) V	(4, 6) V
V07	(1, 5) V	(1, 5) V
V08	(0, 9) V	(0, 9) V
V09	Di luar	Di luar
V10	(7, 1) H	(7, 1) H
V11	Di luar	Di luar
V12	(1, 7) V	(1, 7) V
V13	(2, 0) H	(2, 0) H
V14	Di luar	Di luar
V15	Di luar	Di luar
V16	(5, 7) V	(5, 7) V

Nilai awal 856 juta rupiah; nilai akhir 856 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.17 Penempatan Kendaraan GA_P10 Seed 43

Kendaraan	State Awal	State Akhir
V01	(2, 7) H	(2, 7) H
V02	Di luar	Di luar
V03	(4, 7) H	(4, 7) H
V04	(3, 10) H	(3, 10) H
V05	(4, 3) V	(4, 3) V
V06	Di luar	Di luar
V07	(2, 3) H	(2, 3) H
V08	(0, 6) H	(0, 6) H
V09	Di luar	Di luar
V10	(7, 8) H	(7, 8) H
V11	Di luar	Di luar
V12	(0, 1) V	(0, 1) V
V13	Di luar	Di luar
V14	(3, 6) V	(3, 6) V
V15	Di luar	Di luar
V16	Di luar	Di luar

Nilai awal 870 juta rupiah; nilai akhir 870 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.18 Penempatan Kendaraan GA_P10 Seed 44

Kendaraan	State Awal	State Akhir
V01	(3, 5) V	(3, 5) V
V02	(0, 0) V	(0, 0) V
V03	(4, 7) H	(4, 7) H
V04	(5, 0) H	(5, 0) H
V05	Di luar	Di luar
V06	Di luar	Di luar
V07	Di luar	Di luar
V08	(4, 2) V	(4, 2) V
V09	(0, 4) V	(0, 4) V
V10	(2, 7) H	(2, 7) H
V11	Di luar	Di luar
V12	(0, 8) H	(0, 8) H
V13	(5, 10) V	(5, 10) V
V14	Di luar	Di luar
V15	Di luar	Di luar
V16	Di luar	Di luar

Nilai awal 866 juta rupiah; nilai akhir 866 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.19 Penempatan Kendaraan GA_P30 Seed 42

Kendaraan	State Awal	State Akhir
V01	Di luar	Di luar
V02	(2, 8) V	(2, 8) V
V03	(4, 3) V	(4, 3) V
V04	Di luar	Di luar
V05	Di luar	Di luar
V06	(0, 6) V	(0, 6) V
V07	(1, 4) V	(1, 4) V
V08	(1, 0) V	(1, 0) V
V09	Di luar	Di luar
V10	(7, 2) H	(7, 2) H
V11	(3, 0) H	(3, 0) H
V12	(3, 10) V	(3, 10) V
V13	Di luar	Di luar
V14	(2, 10) H	(2, 10) H
V15	Di luar	Di luar
V16	Di luar	Di luar

Nilai awal 868 juta rupiah; nilai akhir 868 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.20 Penempatan Kendaraan GA_P30 Seed 43

Kendaraan	State Awal	State Akhir
V01	(2, 7) H	(2, 7) H
V02	Di luar	Di luar
V03	(4, 7) H	(4, 7) H
V04	(3, 10) H	(3, 10) H
V05	(4, 3) V	(4, 3) V
V06	Di luar	Di luar
V07	(2, 3) H	(2, 3) H
V08	(0, 6) H	(0, 6) H
V09	Di luar	Di luar
V10	(7, 8) H	(7, 8) H
V11	Di luar	Di luar
V12	(0, 1) V	(0, 1) V
V13	Di luar	Di luar
V14	(3, 6) V	(3, 6) V
V15	Di luar	Di luar
V16	Di luar	Di luar

Nilai awal 870 juta rupiah; nilai akhir 870 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.21 Penempatan Kendaraan GA_P30 Seed 44

Kendaraan	State Awal	State Akhir
V01	(0, 2) V	(0, 2) V
V02	(6, 5) H	(6, 5) H
V03	(2, 9) H	(2, 9) H
V04	(5, 10) H	(5, 10) H
V05	Di luar	Di luar
V06	(3, 2) V	(3, 2) V
V07	(5, 0) V	(5, 0) V
V08	(4, 4) V	(4, 4) V
V09	Di luar	Di luar
V10	Di luar	Di luar
V11	Di luar	Di luar
V12	(0, 6) H	(0, 6) H
V13	(6, 2) V	(6, 2) V
V14	(3, 8) V	(3, 8) V
V15	Di luar	Di luar
V16	Di luar	Di luar

Nilai awal 875 juta rupiah; nilai akhir 875 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.22 Penempatan Kendaraan GA_P60 Seed 42

Kendaraan	State Awal	State Akhir
V01	(3, 3) V	(3, 3) V
V02	(1, 1) V	(1, 1) V
V03	(1, 8) H	(1, 8) H
V04	(4, 7) H	(4, 7) H
V05	Di luar	Di luar
V06	(0, 7) V	(0, 7) V
V07	Di luar	Di luar
V08	Di luar	Di luar
V09	(5, 1) H	(5, 1) H
V10	(7, 3) H	(7, 3) H
V11	Di luar	Di luar
V12	(2, 5) V	(2, 5) V
V13	(1, 3) V	(1, 3) V
V14	(3, 0) V	(3, 0) V
V15	Di luar	Di luar
V16	Di luar	Di luar

Nilai awal 878 juta rupiah; nilai akhir 878 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.23 Penempatan Kendaraan GA_P60 Seed 43

Kendaraan	State Awal	State Akhir
V01	(2, 7) H	(2, 7) H
V02	Di luar	(4, 0) V
V03	(4, 7) H	(4, 7) H
V04	(3, 10) H	(0, 5) V
V05	(4, 3) V	(4, 3) V
V06	Di luar	Di luar
V07	(2, 3) H	(2, 3) H
V08	(0, 6) H	Di luar
V09	Di luar	Di luar
V10	(7, 8) H	(7, 8) H
V11	Di luar	Di luar
V12	(0, 1) V	(0, 1) V
V13	Di luar	Di luar
V14	(3, 6) V	Di luar
V15	Di luar	Di luar
V16	Di luar	Di luar

Nilai awal 870 juta rupiah; nilai akhir 877 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.

Tabel B.24 Penempatan Kendaraan GA_P60 Seed 44

Kendaraan	State Awal	State Akhir
V01	(3, 9) V	(3, 9) H
V02	(4, 0) H	(4, 0) H
V03	(5, 4) H	(5, 4) H
V04	(1, 0) V	(1, 0) V
V05	Di luar	Di luar
V06	(4, 7) V	(4, 7) V
V07	Di luar	(6, 8) H
V08	Di luar	Di luar
V09	(0, 3) V	(0, 3) V
V10	(3, 1) H	(3, 1) H
V11	Di luar	Di luar
V12	(1, 6) H	(1, 6) H
V13	(6, 8) H	Di luar
V14	(1, 10) V	(1, 10) V
V15	Di luar	Di luar
V16	Di luar	Di luar

Nilai awal 878 juta rupiah; nilai akhir 889 juta rupiah. Seluruh kendaraan terangkut pada *state* akhir memenuhi pemeriksaan batas, tumpang tindih, dan kapasitas.